ExtraaLearn Project

Context

The EdTech industry has been surging in the past decade immensely, and according to a forecast, the Online Education market would be worth $286.62bn by 2023 with a compound annual growth rate (CAGR) of 10.26% from 2018 to 2023. The modern era of online education has enforced a lot in its growth and expansion beyond any limit. Due to having many dominant features like ease of information sharing, personalized learning experience, transparency of assessment, etc, it is now preferable to traditional education.

In the present scenario due to the Covid-19, the online education sector has witnessed rapid growth and is attracting a lot of new customers. Due to this rapid growth, many new companies have emerged in this industry. With the availability and ease of use of digital marketing resources, companies can reach out to a wider audience with their offerings. The customers who show interest in these offerings are termed as leads. There are various sources of obtaining leads for Edtech companies, like

- The customer interacts with the marketing front on social media or other online platforms.
- The customer browses the website/app and downloads the brochure
- The customer connects through emails for more information.

The company then nurtures these leads and tries to convert them to paid customers. For this, the representative from the

organization connects with the lead on call or through email to share further details.

Objective

ExtraaLearn is an initial stage startup that offers programs on cutting-edge technologies to students and professionals to help them upskill/reskill. With a large number of leads being generated on a regular basis, one of the issues faced by ExtraaLearn is to identify which of the leads are more likely to convert so that they can allocate resources accordingly. You, as a data scientist at ExtraaLearn, have been provided the leads data to:

- Analyze and build an ML model to help identify which leads are more likely to convert to paid customers,
- Find the factors driving the lead conversion process
- Create a profile of the leads which are likely to convert

Data Description

The data contains the different attributes of leads and their interaction details with ExtraaLearn. The detailed data dictionary is given below.

Data Dictionary

- ID: ID of the lead

- age: Age of the lead

- current_occupation: Current occupation of the lead. Values include 'Professional','Unemployed',and 'Student'

- first_interaction: How did the lead first interacted with ExtraaLearn. Values include 'Website', 'Mobile App'
- profile_completed: What percentage of profile has been filled by the lead on the website/mobile app. Values include Low - (0-50%), Medium - (50-75%), High (75-100%)
- website_visits: How many times has a lead visited the website
- time_spent_on_website: Total time spent on the website
- page_views_per_visit: Average number of pages on the website viewed during the visits.
- last_activity: Last interaction between the lead and ExtraaLearn.
    - Email Activity: Seeking for details about program through email, Representative shared information with lead like brochure of program , etc
    - Phone Activity: Had a Phone Conversation with representative, Had conversation over SMS with representative, etc
    - Website Activity: Interacted on live chat with representative, Updated profile on website, etc
- print_media_type1: Flag indicating whether the lead had seen the ad of ExtraaLearn in the Newspaper.
- print_media_type2: Flag indicating whether the lead had seen the ad of ExtraaLearn in the Magazine.
- digital_media: Flag indicating whether the lead had seen the ad of ExtraaLearn on the digital platforms.
- educational_channels: Flag indicating whether the lead had heard about ExtraaLearn in the education channels like

online forums, discussion threads, educational websites, etc.

- referral: Flag indicating whether the lead had heard about ExtraaLearn through reference.

- status: Flag indicating whether the lead was converted to a paid customer or not.

Importing necessary libraries and data

In [85]:
```
# Importing libraries for data manipulation
import pandas as pd
import numpy as np

# Importing libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# To ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)

# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 200)

# To build models for prediction
from sklearn.model_selection import train_test_split, cross_
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, Bagging
from sklearn.linear_model import LogisticRegression

# To encode categorical variables
from sklearn.preprocessing import LabelEncoder

# For tuning the model
from sklearn.model_selection import GridSearchCV, Randomized
```

```
# To check model performance
from sklearn.metrics import (
    make_scorer,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay,
)
```

In [86]:
```
# Read the healthcare dataset file
original_data = pd.read_csv("/content/ExtraaLearn PotentialC
```

In [87]:
```
# Copying data in case we need the original dataset
data = original_data.copy()
```

Data Overview

- Observations
- Sanity checks

In [88]:
```
# View the first 5 rows of the dataset
data.head()
```

Out[88]:

	ID	age	current_occupation	first_interaction	profile_comp
0	EXT001	57	Unemployed	Website	
1	EXT002	56	Professional	Mobile App	M
2	EXT003	52	Professional	Website	M
3	EXT004	53	Unemployed	Website	
4	EXT005	23	Student	Website	

In [89]:

```
# Understand the shape of the data
data.shape
```

Out[89]:

```
(4612, 15)
```

- The dataset has **4612 rows and 15 columns.**

In [90]:

```
# Checking the info/columns of the data
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4612 entries, 0 to 4611
Data columns (total 15 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   ID                     4612 non-null   object
 1   age                    4612 non-null   int64
 2   current_occupation     4612 non-null   object
 3   first_interaction      4612 non-null   object
 4   profile_completed      4612 non-null   object
 5   website_visits         4612 non-null   int64
 6   time_spent_on_website  4612 non-null   int64
 7   page_views_per_visit   4612 non-null   float64
 8   last_activity          4612 non-null   object
 9   print_media_type1      4612 non-null   object
 10  print_media_type2      4612 non-null   object
 11  digital_media          4612 non-null   object
 12  educational_channels   4612 non-null   object
 13  referral               4612 non-null   object
 14  status                 4612 non-null   int64
dtypes: float64(1), int64(4), object(10)
memory usage: 540.6+ KB
```

Observations:

- Age, website_visits, time_spent_on_website, page_views_per_visit, and status are of **numeric data type**
- The columns **ID**, **first_interaction**, **current_ocupation**, **profiled_completed**, **last_activity**, **print_media_type1**, **print_media_type2**, **digital_media**, **educational_channels** and referral have data type **object**, which means they are strings or categorical variables.
- The number of non-null values is the same as the total number of entries in the data, **there are no null values.**
- The column ID is an identifier for leads in the data. This column will not help with our analysis so we can drop it.

In [91]:
```
# Checking the number of unique values in each column
data.nunique()
```

Out[91]:

	0
ID	4612
age	46
current_occupation	3
first_interaction	2
profile_completed	3
website_visits	27
time_spent_on_website	1623
page_views_per_visit	2414
last_activity	3
print_media_type1	2
print_media_type2	2
digital_media	2
educational_channels	2
referral	2
status	2

dtype: int64

Most of the categorical variables are YES/NO columns. Maybe we can convert them to 0 and 1.

In [92]:

```
# To view ID and the number of times each lead is in the dat
data['ID'].value_counts()
```

Out[92]:

ID	count
EXT4612	1
EXT001	1
EXT002	1
EXT003	1
EXT4596	1
...	...
EXT009	1
EXT008	1
EXT007	1
EXT006	1
EXT005	1

4612 rows × 1 columns

dtype: int64

Observation:

The maximum number of times the same ID is in the dataset is 1, but we need to verify the rest of the columns, since the correlative in this case can be different, but the rest of variables can be the same.

In [93]:
```
# Dropping ID of the lead from the data as it is an identifi
data=data.drop(columns=["ID"])
```

In [94]:
```
# Checking for duplicate values in the data
```

```
data.duplicated().sum()
```

Out[94]: np.int64(14)

Observation:

- Data contains 14 duplicate rows. There is need to remove those rows.

In [95]:
```
#Clean duplicate rows
data=data.drop_duplicates()
```

In [96]:
```
#New shape of the data after dropping duplicates
data.shape
```

Out[96]: (4598, 14)

Observation:

- No duplicates are now in the data
- Now rows are 4598 and columns 14

In [97]:
```
# Creating numerical columns
num_cols = ['age', 'website_visits', 'time_spent_on_website'
```

In [98]:
```
# Checking the descriptive statistics of the numerical colum
data.describe().T
```

Out[98]:

	count	mean	std	min	25
age	4598.0	46.179208	13.160811	18.0	36.(
website_visits	4598.0	3.577642	2.826572	0.0	2.(
time_spent_on_website	4598.0	726.215746	743.884963	0.0	155.2
page_views_per_visit	4598.0	3.035339	1.964011	0.0	2.(
status	4598.0	0.299043	0.457889	0.0	0.(

Observations:

- **On average, the lead age is 46 years old**, mininum age is 18 and maximum is 63.
- **There are 3 visits to the website in average per lead**, but there are leads that have not been in the website (min 0), and there are others that have visited the website 30 times!.
- The time spent on the website is in seconds, so the mean time is 12 minutes and max time is 42 minutes after doing the convertion. I will have to convert this colum later.
- **On average the pages views are 3**, with a maximum of 18 page views per visit.
- On average, the status of the leads have a min of 0 and a maximum of 1. ***The status column represent the unpaid vs the paid customers and it is our target variable***.

In [99]:

```
# Creating categorical variables
cat_cols = ["current_occupation","first_interaction", 'profi
            'digital_media', "educational_channels", "referra
```

In [100…

```
# Printing the number of occurrences of each unique value in
for column in cat_cols:
    print(data[column].value_counts(1))
    print("-" * 50)
```

```
current_occupation
Professional    0.567203
Unemployed      0.312310
Student         0.120487
Name: proportion, dtype: float64
--------------------------------------------------
first_interaction
Website       0.551544
Mobile App    0.448456
Name: proportion, dtype: float64
--------------------------------------------------
profile_completed
High      0.490431
Medium    0.486298
Low       0.023271
Name: proportion, dtype: float64
--------------------------------------------------
last_activity
Email Activity      0.493475
Phone Activity      0.267290
Website Activity    0.239234
Name: proportion, dtype: float64
--------------------------------------------------
print_media_type1
No     0.892127
Yes    0.107873
Name: proportion, dtype: float64
--------------------------------------------------
print_media_type2
No     0.949326
Yes    0.050674
Name: proportion, dtype: float64
--------------------------------------------------
digital_media
No     0.885385
Yes    0.114615
Name: proportion, dtype: float64
--------------------------------------------------
educational_channels
No     0.84689
Yes    0.15311
Name: proportion, dtype: float64
--------------------------------------------------
```

```
referral
No     0.979774
Yes    0.020226
Name: proportion, dtype: float64
--------------------------------------------------
```

Observations:

- **The majority of leads (~55%) are professionals**, which shows there are lots of professionals looking to upskill or reskill.
- **The first interaction of 55% of the leads was through the website**, which is not surprising since we are in a digital era and the people do not download or visit an application unless they are going to use it often.
- **The profile completion is acceptable in general 98%**, 49% having high completion, ~49% medium level, and there are very few (2%) in low range whose have not completed their profile.
- **The last activity of (~50%) was by email**, this is a point to analyze to understand if this represent the % that is already converted to a paid customer. The Website activity and phone activity is pretty similar, being around the 24% and 27% respectively.
- The print_media_type1/newspaper sourced has a 10%, the print_media_type2/magazine sourced has 5%, the digital_media platforms as sourced of leads contribute with an 11%, the educational_channels with a 15%, and referral with just 2%. All of this percentages are in terms of the total quantity of leads per variable, treated as independent variables. So those variables such as print_media_type1, print_media_type2, digital_media and educational_channels need to be analysed together, to understand which is the percentage of leads that belong to each source taking in

consideration the big picture/marketing leads. We do not know if they are mutually excluding the rest of them.

Exploratory Data Analysis (EDA)

- EDA is an important part of any project involving data.
- It is important to investigate and understand the data better before building a model with it.
- A few questions have been mentioned below which will help you approach the analysis in the right manner and generate insights from the data.
- A thorough analysis of the data, in addition to the questions mentioned below, should be done.

Questions

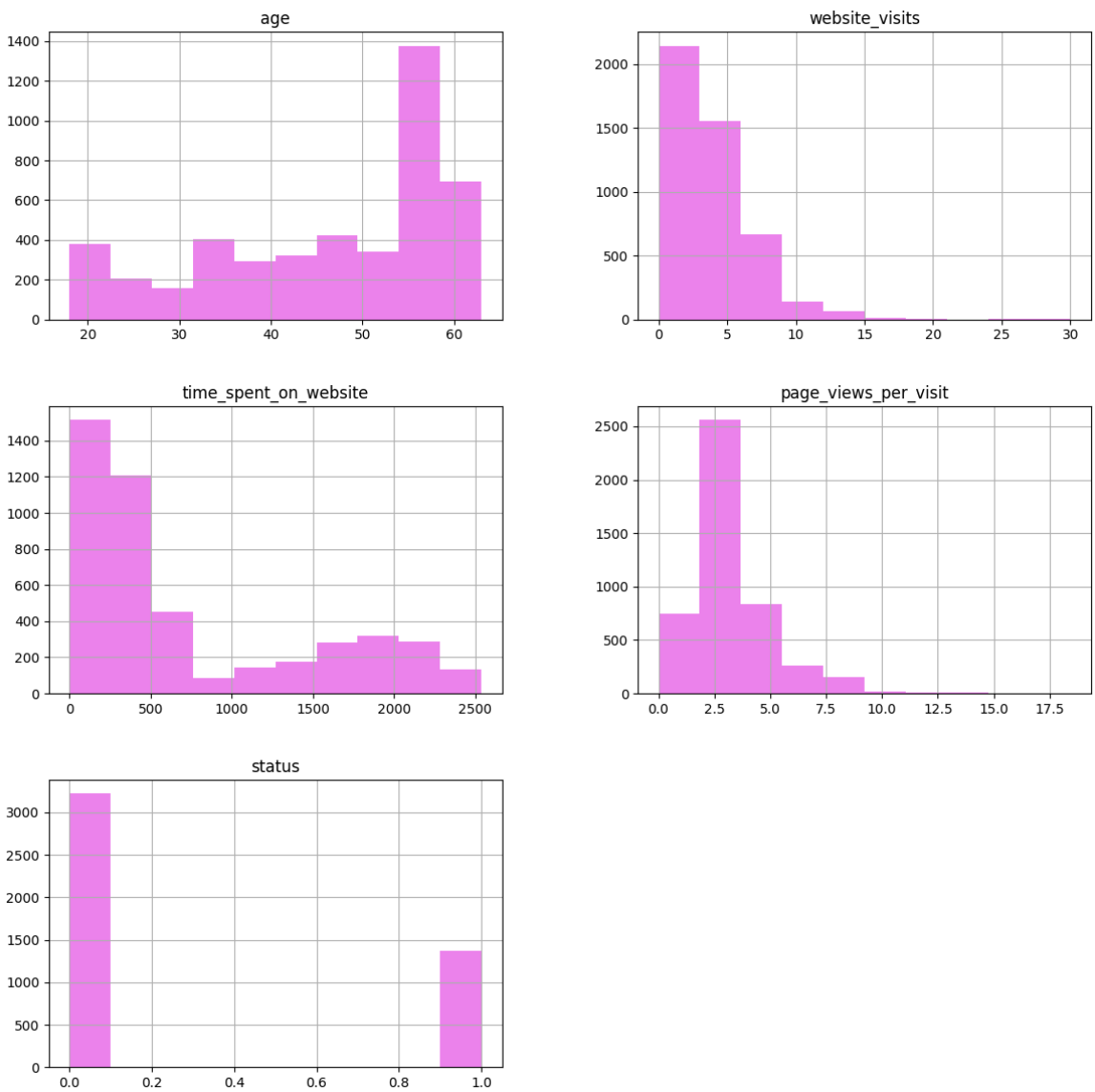
1. Leads will have different expectations from the outcome of the course and the current occupation may play a key role in getting them to participate in the program. Find out how current occupation affects lead status.
2. The company's first impression on the customer must have an impact. Do the first channels of interaction have an impact on the lead status?
3. The company uses multiple modes to interact with prospects. Which way of interaction works best?
4. The company gets leads from various channels such as print media, digital media, referrals, etc. Which of these channels have the highest lead conversion rate?
5. People browsing the website or mobile application are generally required to create a profile by sharing their personal data before they can access additional information.Does having more details about a prospect increase the chances of conversion?

Univariate Analysis

Numerical Variables

In [101…

```
#Histogram for numerical columns
hist_axes = data[num_cols].hist(figsize = (14, 14))
for row_axes in hist_axes:
    for ax in row_axes:
        for patch in ax.patches:
            patch.set_facecolor('violet')
plt.show()
```



Observations:

- The age distribution is skewed to the left, with a peak around 50 years old, but there is still a good portion of leads spread between the range of the 20s to the 50s.
- Website_visits and Pages_viewed_per_visit are right-skewed distribution, meaning most of the leads visit the website few times and do not visit lots of pages in their visits.
- Time_spent_on_website is like a bimodal distribution, where some leads are around 240 seconds and other group of leads is around 2000 seconds, maybe this is the difference between the unpaid and paid customers.
- ***Status is a Binomial distribution, indicating that the majority of the leads, more than 3,000, were not converted to paid customers, and just around 1,350 got converted***.

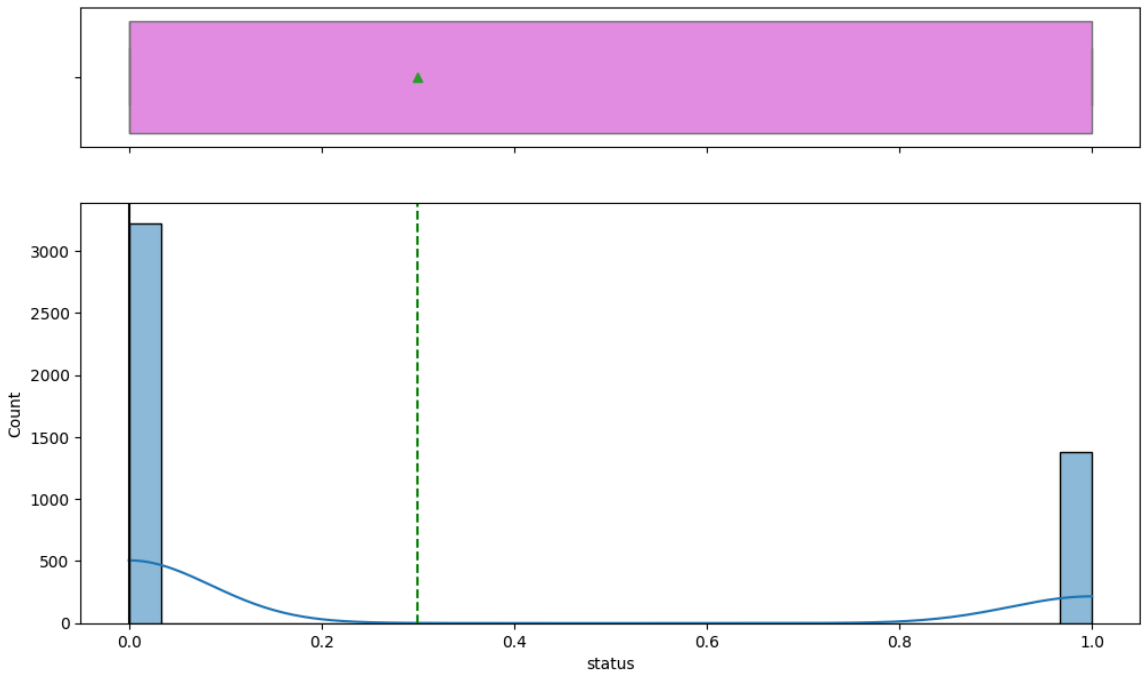
In [102…

```
# Function to plot a boxplot to visualize outliers and a his

def histogram_boxplot(data, feature, figsize=(12, 7), kde=Fa
    """
    Boxplot and histogram combined

    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (12,7))
    kde: whether to the show density curve (default False)
    bins: number of bins for histogram (default None)
    """
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows = 2,       # Number of rows of the subplot gric
        sharex = True,   # x-axis will be shared among all su
        gridspec_kw = {"height_ratios": (0.25, 0.75)},
        figsize = figsize,
    )                    # Creating the 2 subplots
    sns.boxplot(data = data, x = feature, ax = ax_box2, show
    )                    # Boxplot will be created and a star
    sns.histplot(
        data = data, x = feature, kde = kde, ax = ax_hist2,
    ) if bins else sns.histplot(
```

```
        data = data, x = feature, kde = kde, ax = ax_hist2
    )                   # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color = "green", linestyle = '
    )                   # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color = "black", linestyle =
    )                   # Add median to the histogram
```

In [103…

```
#To visualize current Lead Status distribution
histogram_boxplot(data, "status", kde = True, bins = 30)
```



In [104…

```
#To check how many leads have not converted to paid customer
data['status'].value_counts()
```

Out[104…

status	count
0	3223
1	1375

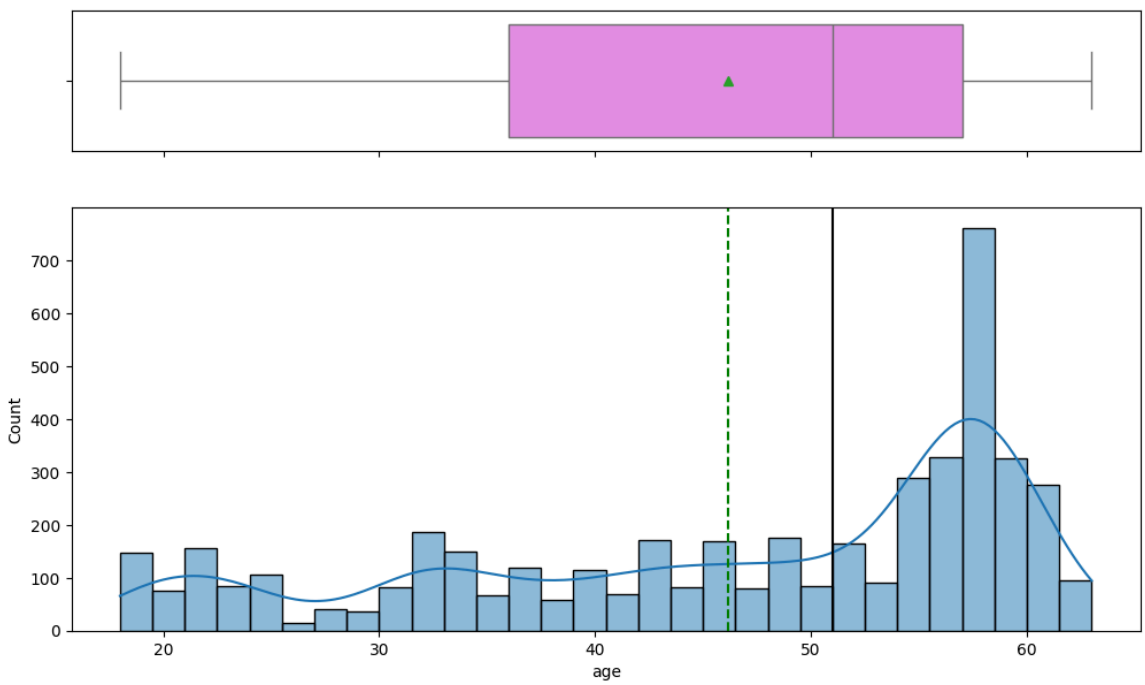
dtype: int64

Observations:

- **This is a binary distribution where the big majority of the leads, more than 3000 in fact, are not converted to paid customers, and just around 1375 leads were converted.**
- There is no other option here, a lead will convert or not convert, which translate a ***classification problem.***

In [105…

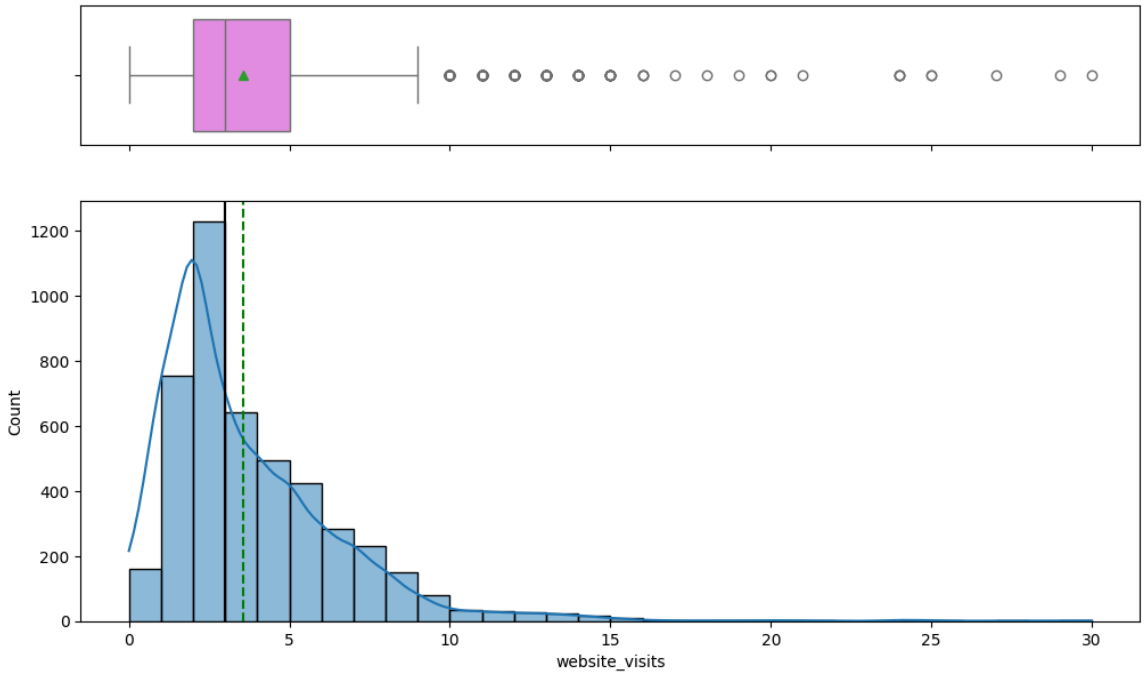
```
# Visualize Age
histogram_boxplot(data, "age", kde = True, bins = 30)
```



Skewed to the left. Lots of the leads are above their 50s. The mean is around 45 years old, but there are younger people, all adults from 18 years to up, interested in Edtech online education.

In [106…

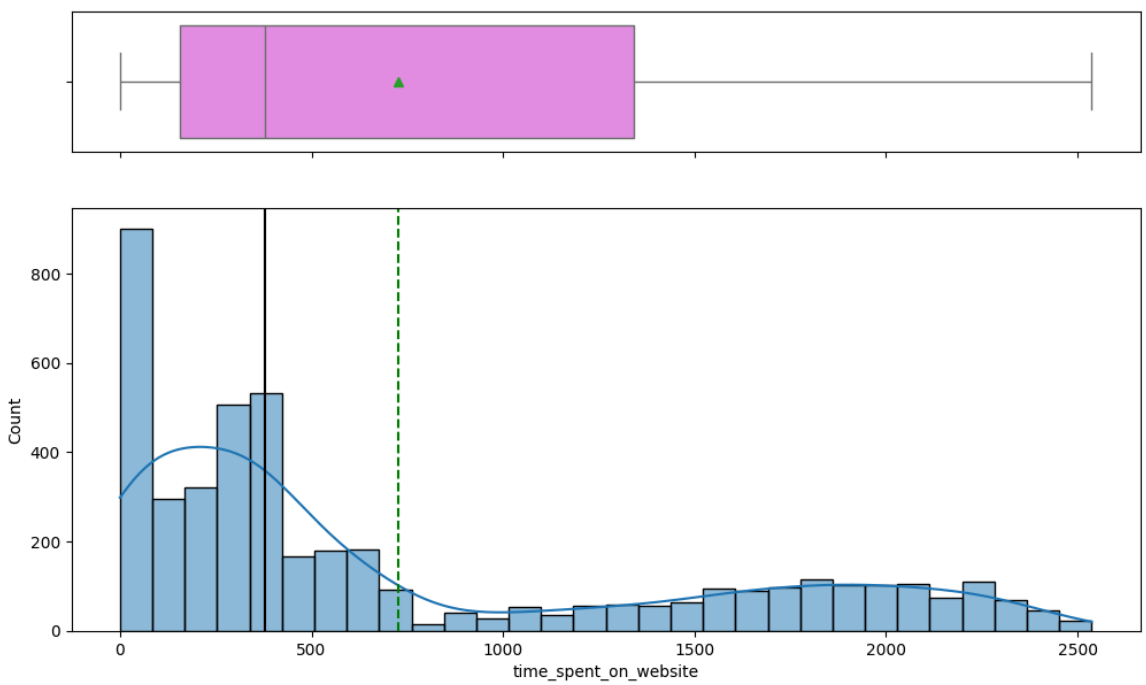
```
# Visualize the Website visits
histogram_boxplot(data, "website_visits", kde = True, bins =
```



Skewed to the right. **The majority of the leads have visited the website less than 10 times** and the peak of the distribution shows website visits between 3 to 4 times in average. There are some outliers that have visited the site more that 10 times.

In [107…

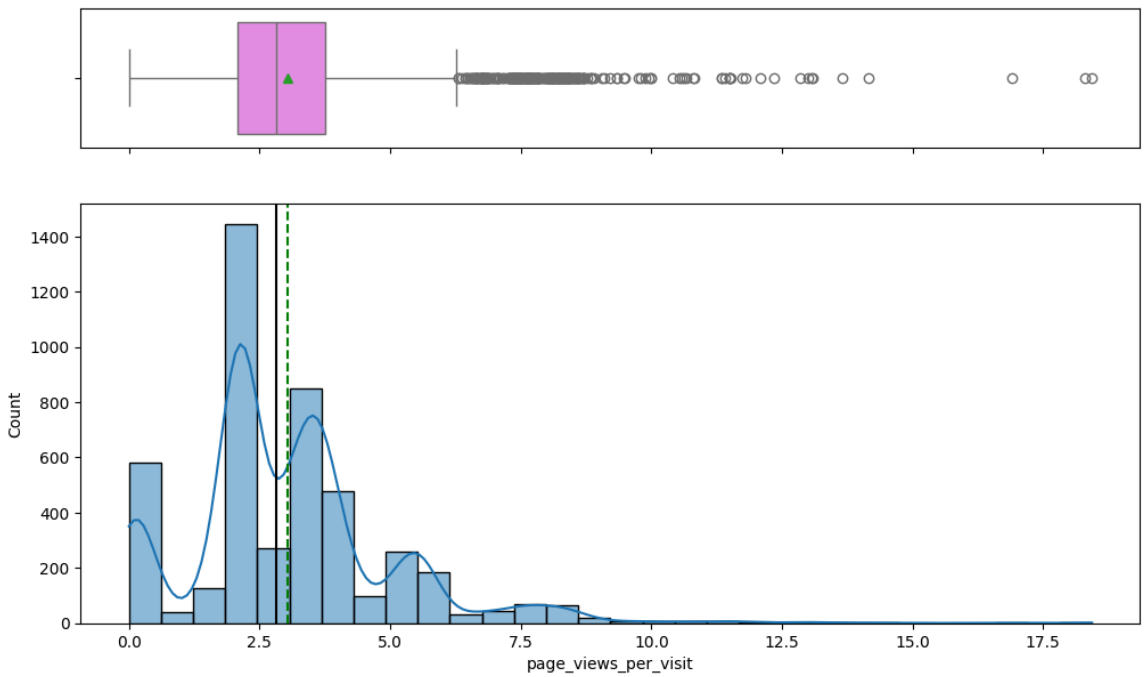
```
# Visualize the Time Spend on the website
histogram_boxplot(data, "time_spent_on_website", kde = True,
```



Interesting distribution where there are 2 clear areas very distance. **The distribution shows leads that spent up to ~2500 seconds/42 minutes**, but in average the time in the website per lead is ~750 seconds/12 minutes. I need to analyze this variable in conjuction with another one to better understand this distribution behaviour.

In [108…

```
# Vizualize pages view per visit
histogram_boxplot(data, "page_views_per_visit", kde = True,
```



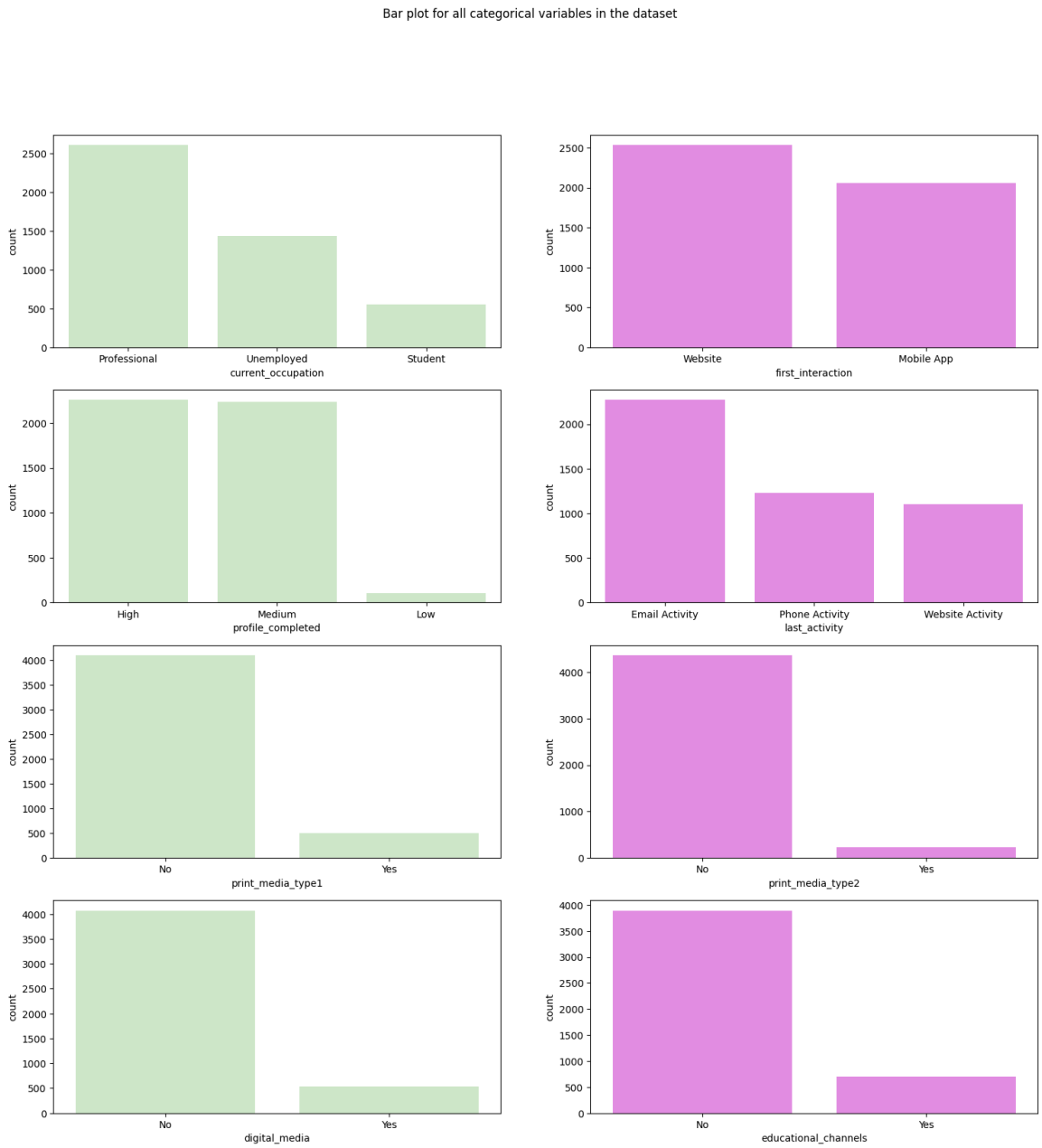
Long tail to the right shows **lots of outliers of the variable page_views_per_visit that go aproximately from 6 up to 18**. Mean is around 3 pages viewed per visit, but there is a peak in the curve that shows around 1400 leads that have just visited ~2 pages.

Categorical Variables

Below we are creating subplots to plot the categorical variables in a single frame.

In [109…

```
# To create a block of subplots
fig, axes = plt.subplots(4, 2, figsize = (18, 18))

fig.suptitle('Bar plot for all categorical variables in the

sns.countplot(ax = axes[0, 0], x = 'current_occupation', dat
              order = data['current_occupation'].value_count

sns.countplot(ax = axes[0, 1], x = 'first_interaction', data
              order = data['first_interaction'].value_counts

sns.countplot(ax = axes[1, 0], x = 'profile_completed', data
              order = data['profile_completed'].value_counts

sns.countplot(ax = axes[1, 1], x = 'last_activity', data = d
              order = data['last_activity'].value_counts().i

sns.countplot(ax = axes[2, 0], x = 'print_media_type1', data
              order = data['print_media_type1'].value_counts

sns.countplot(ax = axes[2, 1], x = 'print_media_type2', data
              order = data['print_media_type2'].value_counts

sns.countplot(ax = axes[3, 0], x = 'digital_media', data = d
              order = data['print_media_type1'].value_counts

sns.countplot(ax = axes[3, 1], x = 'educational_channels', d
              order = data['print_media_type2'].value_counts
```

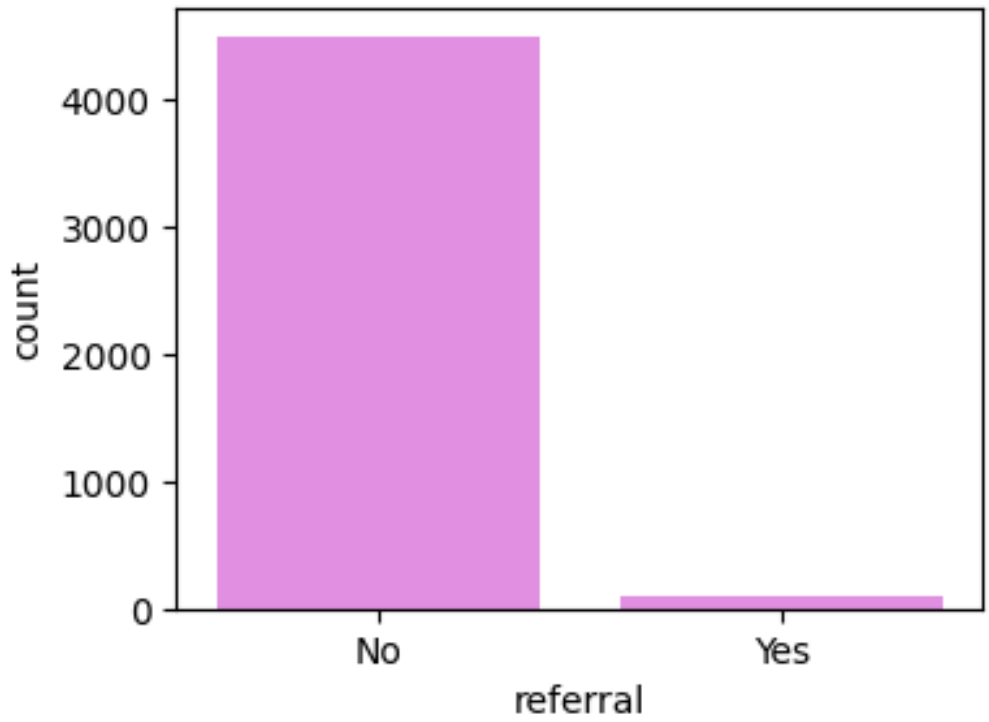


In [110…

```
# Graph for referral because it could not be included in the
plt.figure(figsize = (4,3))
sns.countplot(x = 'referral', data = data, color ='violet')
```

Out[110…

```
<Axes: xlabel='referral', ylabel='count'>
```



Observations in reference to the subplots and the previous data count and proportions calculated for categorical feautures:

- Most of the leads are professionals ~2,500 (56%).
- The first interaction by website ~2,500 (55%) is slighly higher than the mobile app interaction ~2,000 (45%).
- The profile completion is between and high and medium. There is a very small portion of profiles with low completion in the data.
- The last activity is higher by Email (49%) than it is by phone (27%) or website activity (24%). ***The email way of interaction is more common since more than 2000 leads prefer it*** over the phone interaction which has ~1250 and email with ~1000 leads.
- The origin of the leads ***Printed_media_type 1, printed_media_type2, digital_media, educational_channels and referral have very low chances of convertion to paid customers***

- The referral convertion rate is very very low around 2% with only 93 leads converted out of the total of leads 4,598.

Bivariate Analysis

We have analyzed different categorical and numerical variables alone. Let's now check how does STATUS is related with other numerical and categorical variables

Relationship between STATUS and Numerical variables

In [111…

```
# The mean of numerical variables grouped by STATUS
data.groupby(['status'])[num_cols].mean()
```

Out[111…

status	age	website_visits	time_spent_on_website	page_vie
0	45.125349	3.592305	579.570586	
1	48.649455	3.543273	1069.952000	

Observations:

- There is no real difference between the leads with average age converted to paid customer ~45 years old, and the leads that did not convert with average age ~48yo.
- Similar situation happens between the website_visits and page_views_per_visit variables, it seems that the outliers are not affecting that much. **Both variables shows very similar quantity of visits in average ~3.5, and almost the same page_views_per_visit ~3.03.**

- **The only variable that makes the difference here is the time_spent_on_website. Leads converted to paid customers spent almost double the time in the website than the leads that did not convert to paid customers**. It seems that the time invested is related with the deep interest of the customers and its further enrollment.

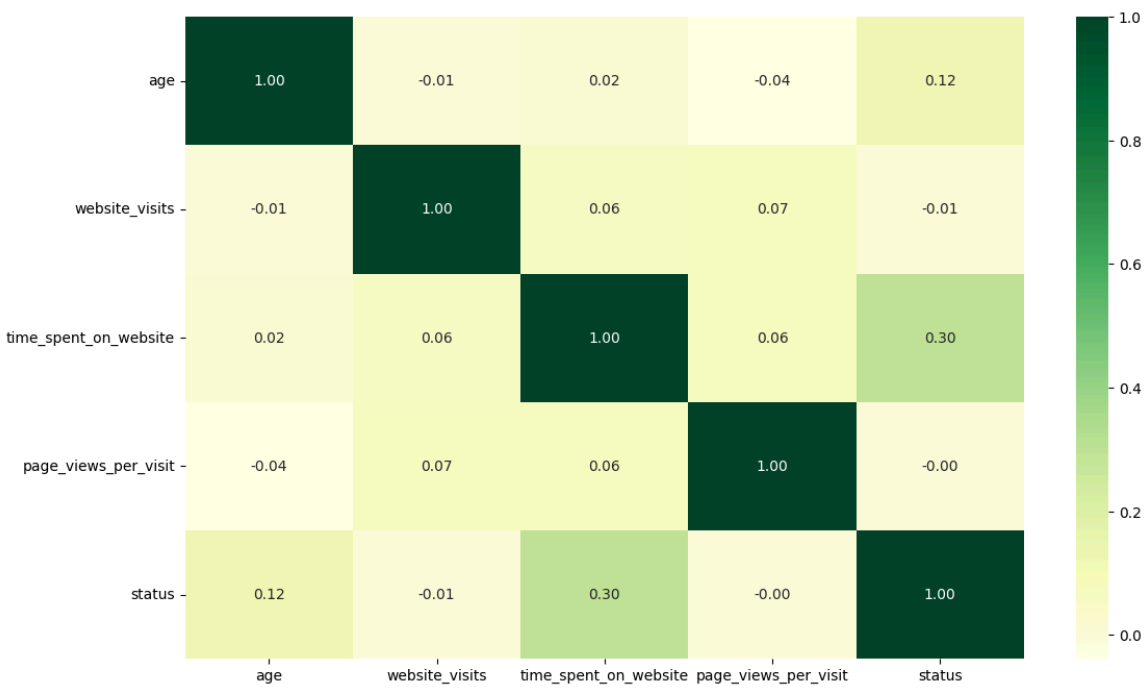
Relationship between Numerical variables

In [112…

```
# Plotting the correlation between numerical variables
plt.figure(figsize = (13, 8))

sns.heatmap(data[num_cols].corr(), annot = True, fmt = '0.2f
```

Out[112…

```
<Axes: >
```



Observations:

- **Confirming the previous observation, the Time spent of website and status are the only variables are slighly correlated with each other**, and all of the rest numerical variables are not really correlated with the resulting status of the leads.

- The numerical variables like age show little or no correlation with the target variable (Status), which would indicate that the categorical variables might be more important for the prediction.

Other numerical functions

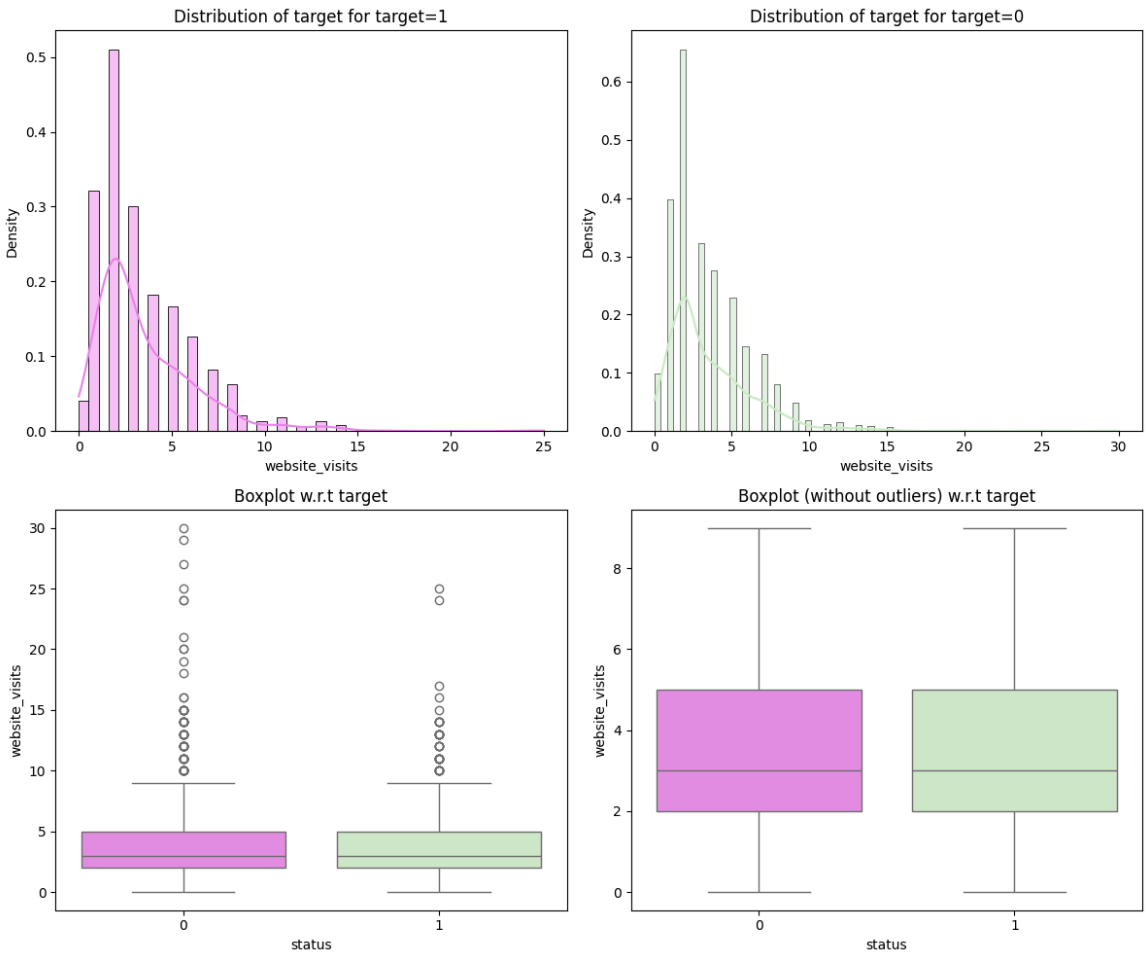
In [113…

```
# function to plot distributions wrt target

def distribution_plot_wrt_target(data, predictor, target):

    fig, axs = plt.subplots(2, 2, figsize=(12, 10))
    target_uniq = data[target].unique()
    axs[0, 0].set_title("Distribution of target for target="
    sns.histplot(
        data=data[data[target] == target_uniq[0]],
        x=predictor,
        kde=True,
        ax=axs[0, 0],
        color="violet",
        stat="density",
    )

    axs[0, 1].set_title("Distribution of target for target="
    sns.histplot(
        data=data[data[target] == target_uniq[1]],
        x=predictor,
        kde=True,
        ax=axs[0, 1],
        color="#ccebc5",
        stat="density",
    )

    axs[1, 0].set_title("Boxplot w.r.t target")
    sns.boxplot(data=data, x=target, y=predictor, ax=axs[1,

    axs[1, 1].set_title("Boxplot (without outliers) w.r.t ta
    sns.boxplot(
        data=data,
        x=target,
        y=predictor,
```

```
        ax=axs[1, 1],
        showfliers=False,
        palette= ["violet", "#ccebc5"],
    )

    plt.tight_layout()
    plt.show()
```

In [114…

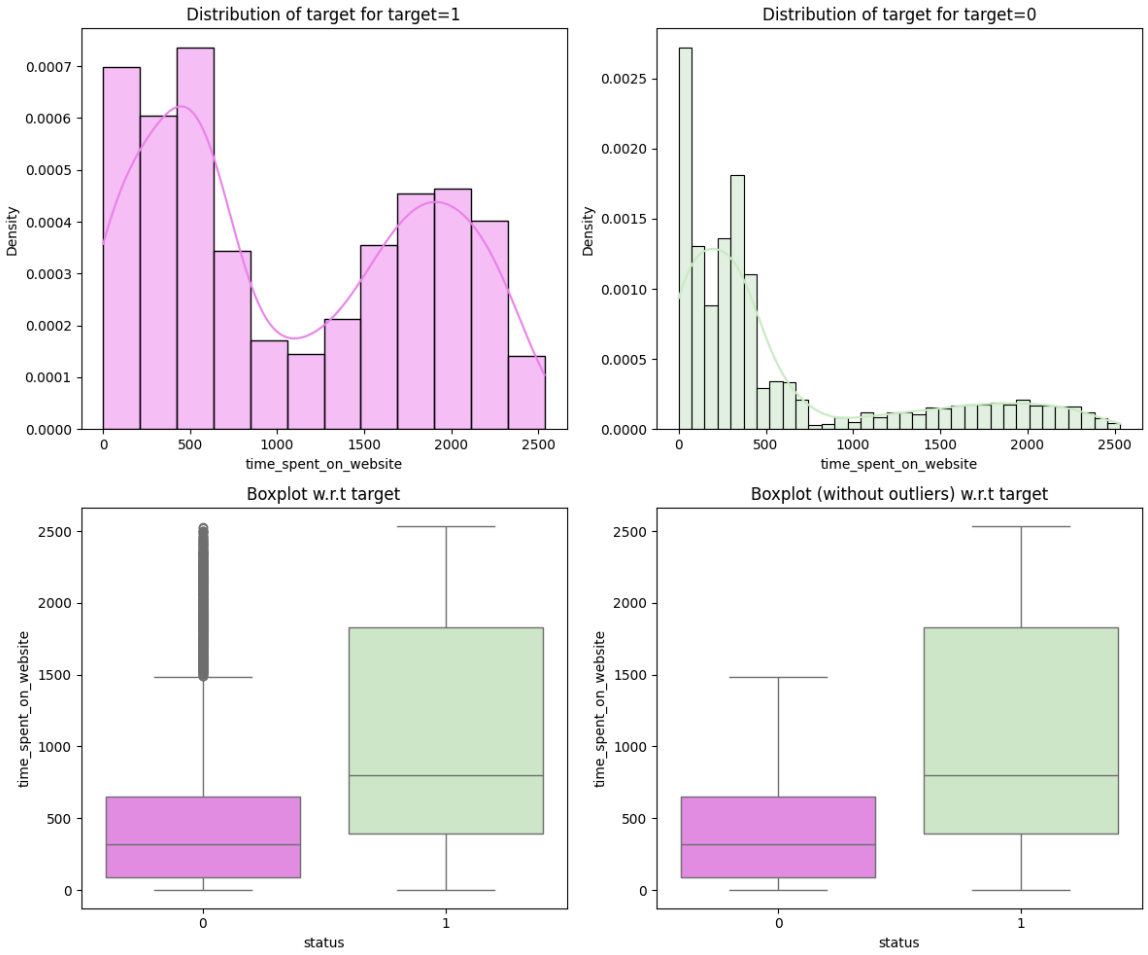
```
distribution_plot_wrt_target(data, "website_visits", "status
```



The number of website_visits is very similar in both status even when there is outliers presence. We already saw a similar conclussion in the table of mean of the numerical variables vs status

In [115…

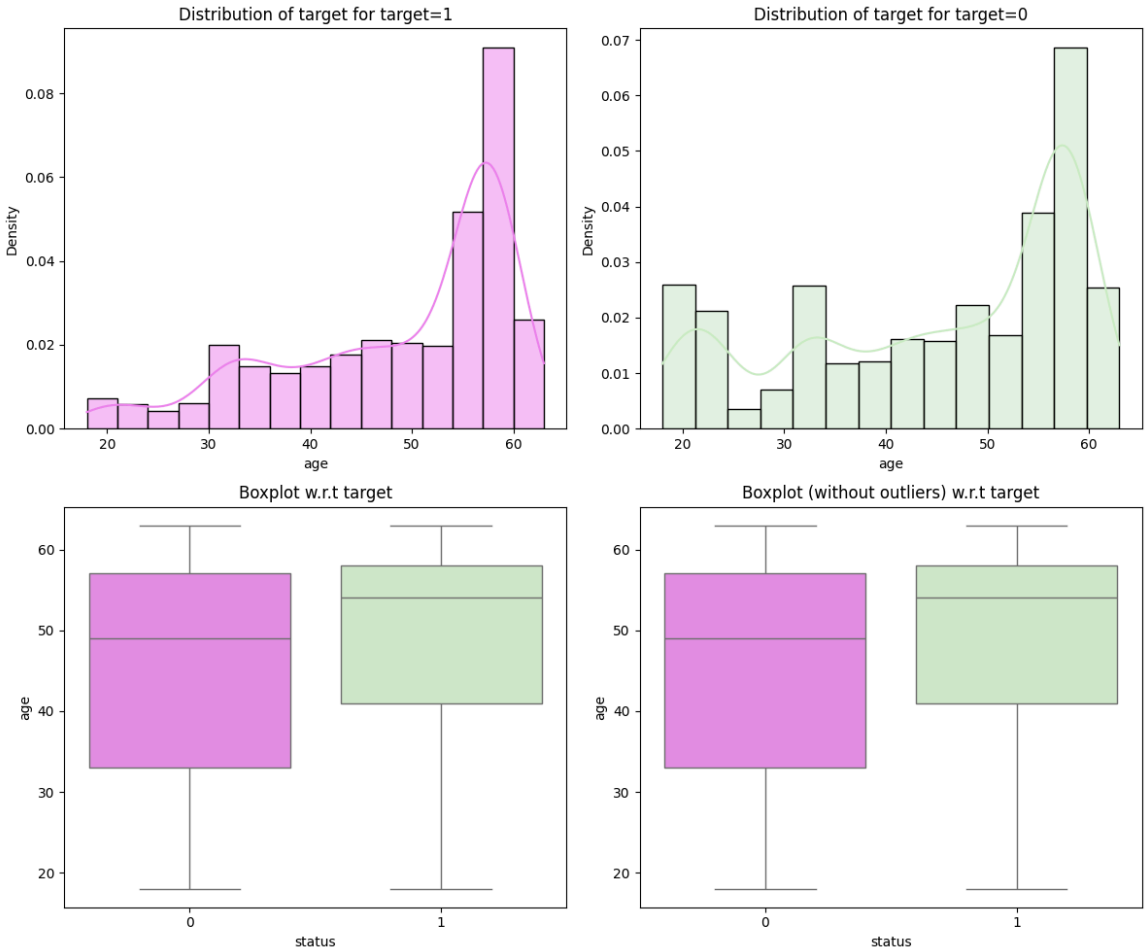
```
distribution_plot_wrt_target(data, "time_spent_on_website",
```



This graph is very interesting, it shows that the paid customers come from higher times spent on the website.

In [116…

```
distribution_plot_wrt_target(data, "age", "status")
```

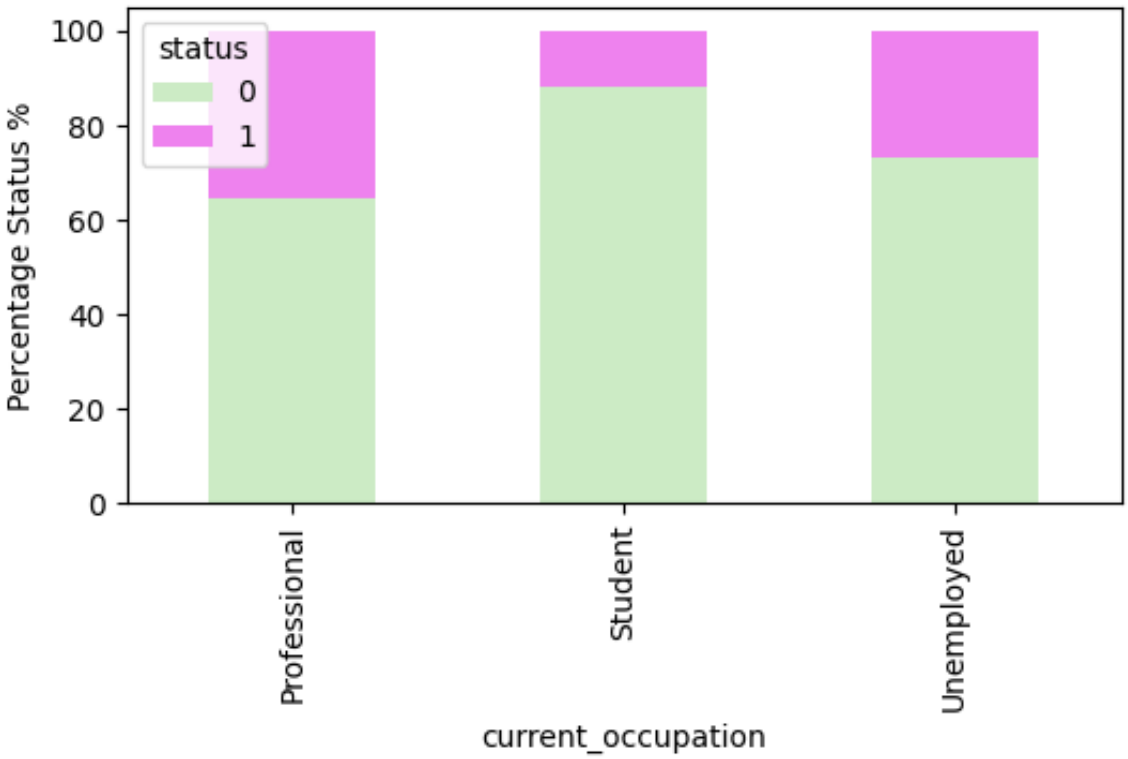


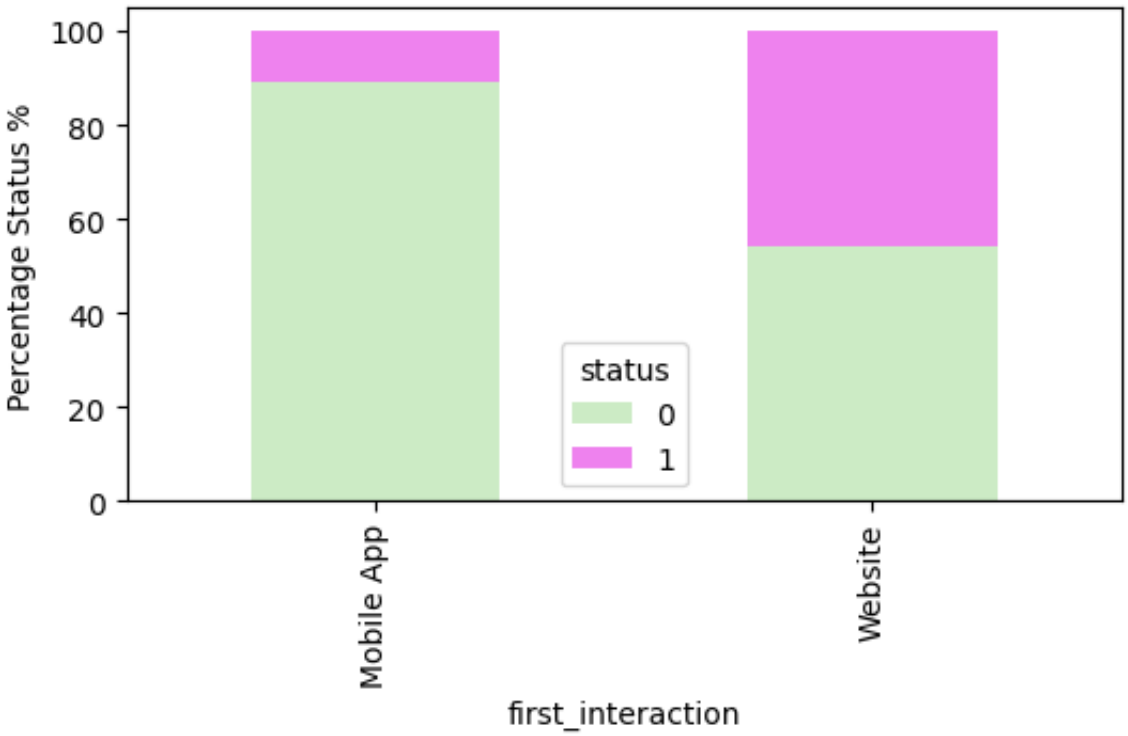
Clearly above than 40 years old the leads are more willing to enroll than younger leads, the mean in fact is approximately 53 years old. The age variable can be improved to focus in converting younger populations as well.

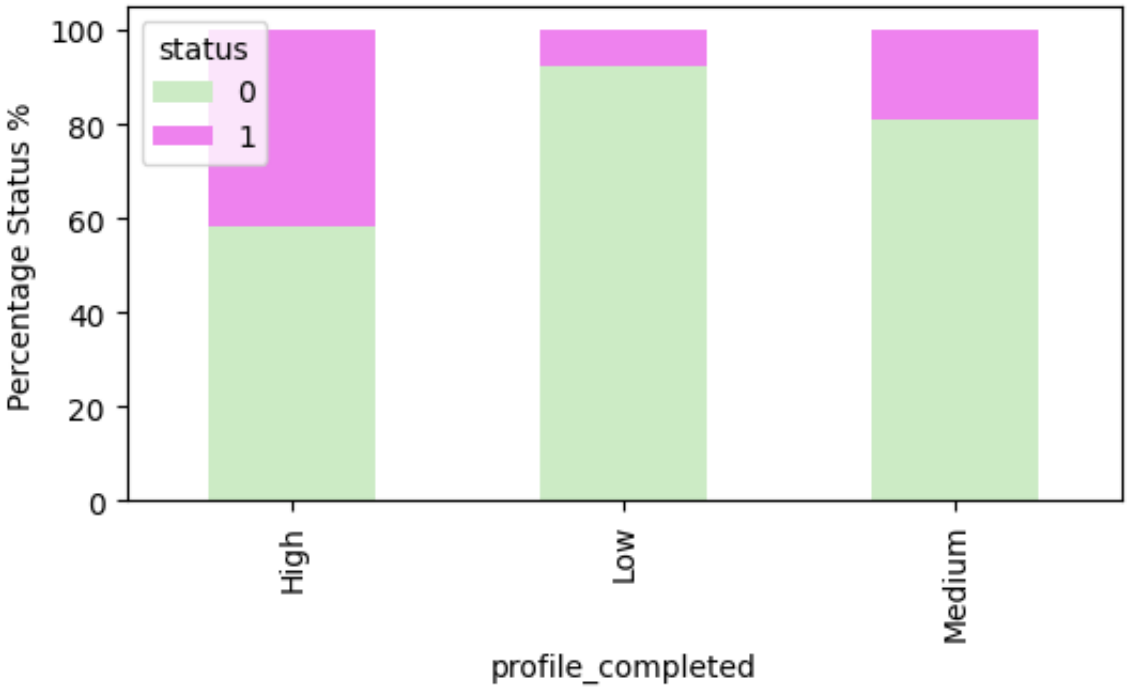
Relationship between STATUS and Categorical variables

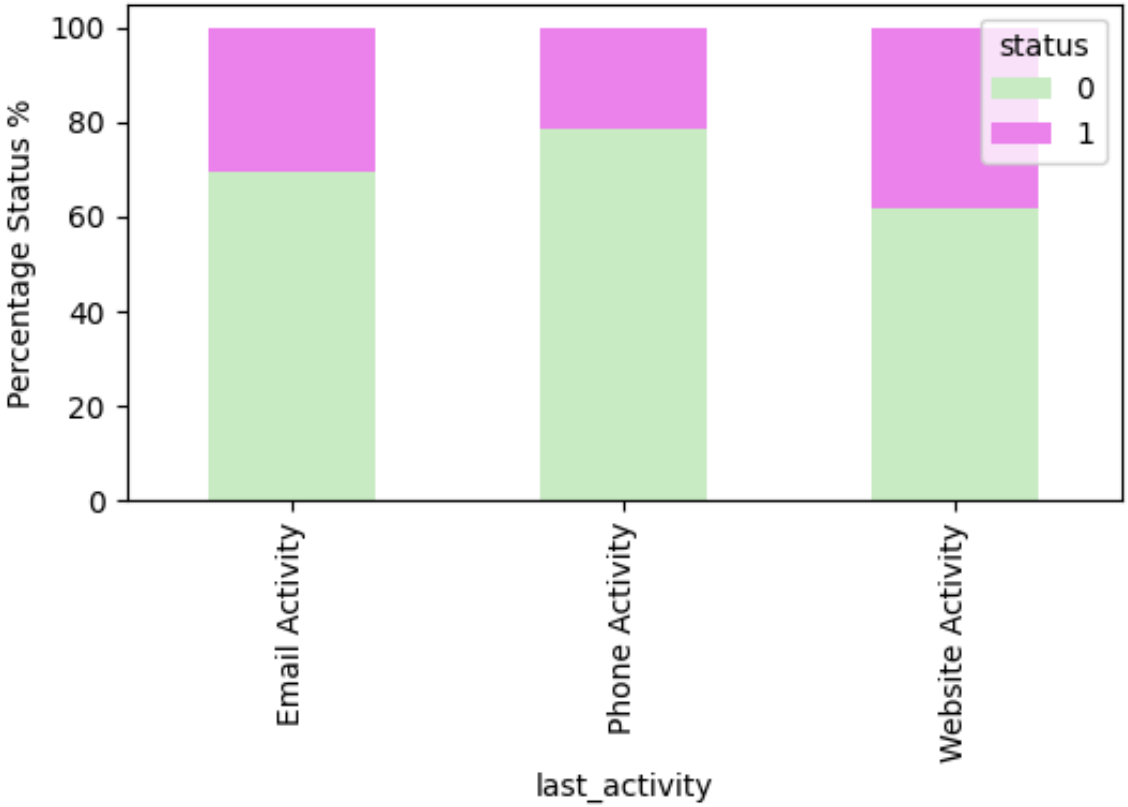
In [117…

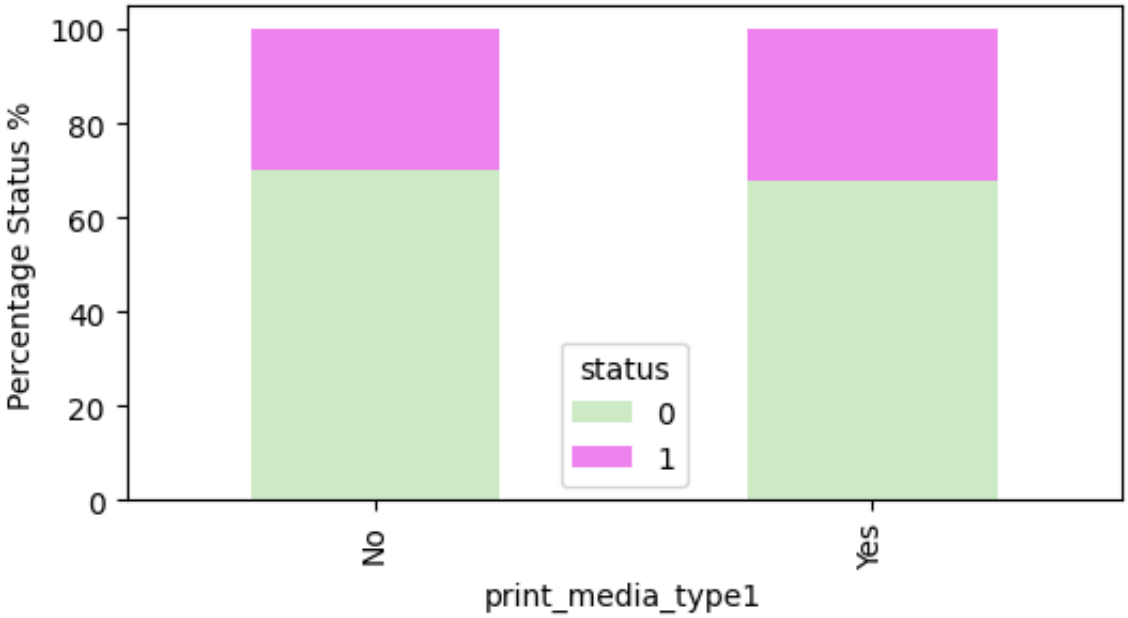
```
for i in cat_cols:
    if i != 'status':
        (pd.crosstab(data[i], data['status'], normalize = 'i
        color=['#ccebc5','violet'])
        plt.ylabel('Percentage Status %')
```

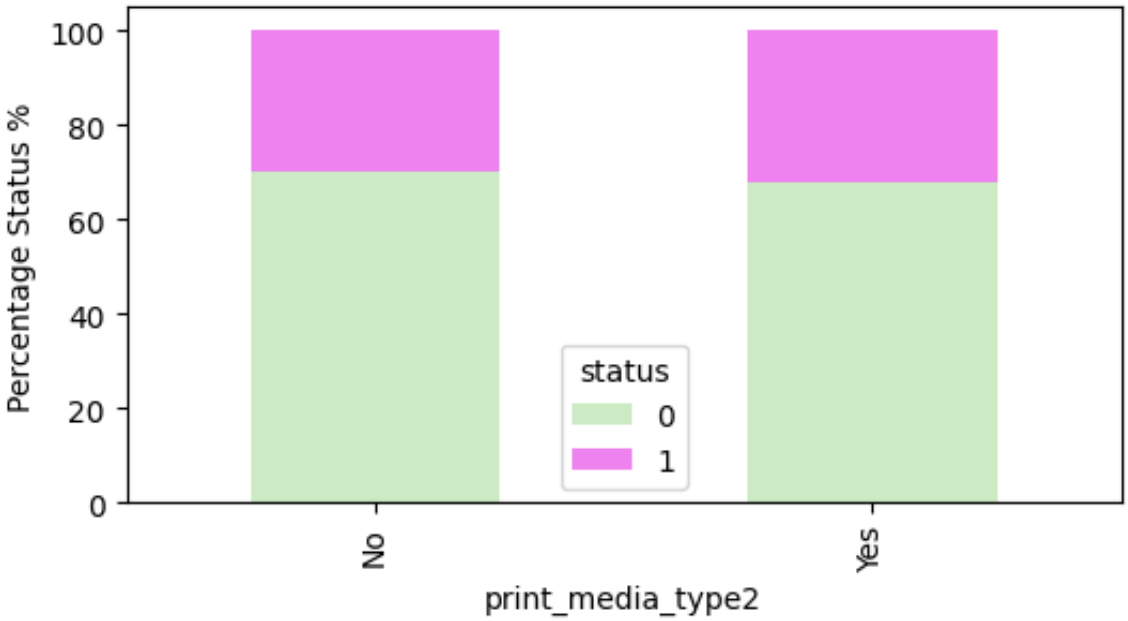

status
0
1
Percentage Status %
100
80
60
40
20
0
Professional
Student
Unemployed
current_occupation

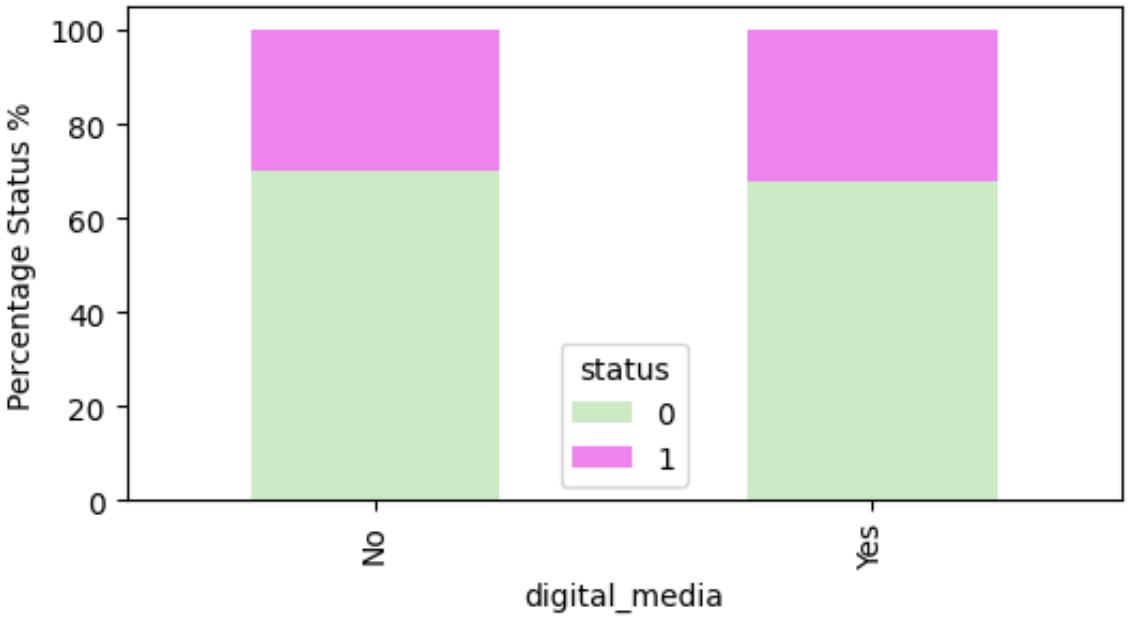

status
0
1
Percentage Status %
100
80
60
40
20
0
Mobile App
Website
first_interaction

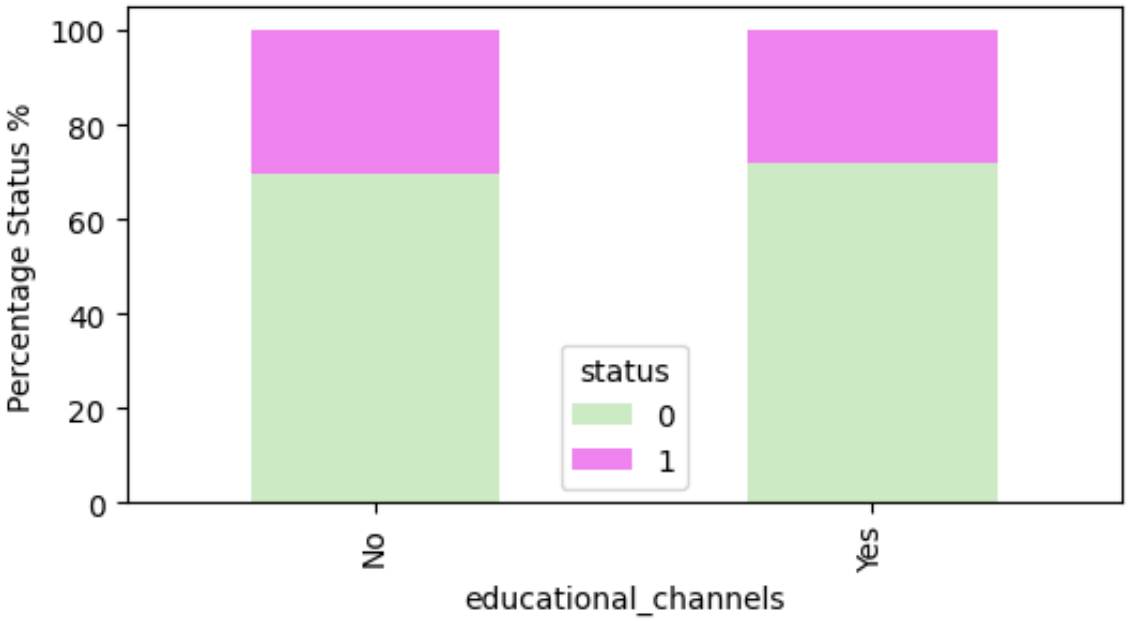

status
0
1
Percentage Status %
100
80
60
40
20
0
High
Low
Medium
profile_completed

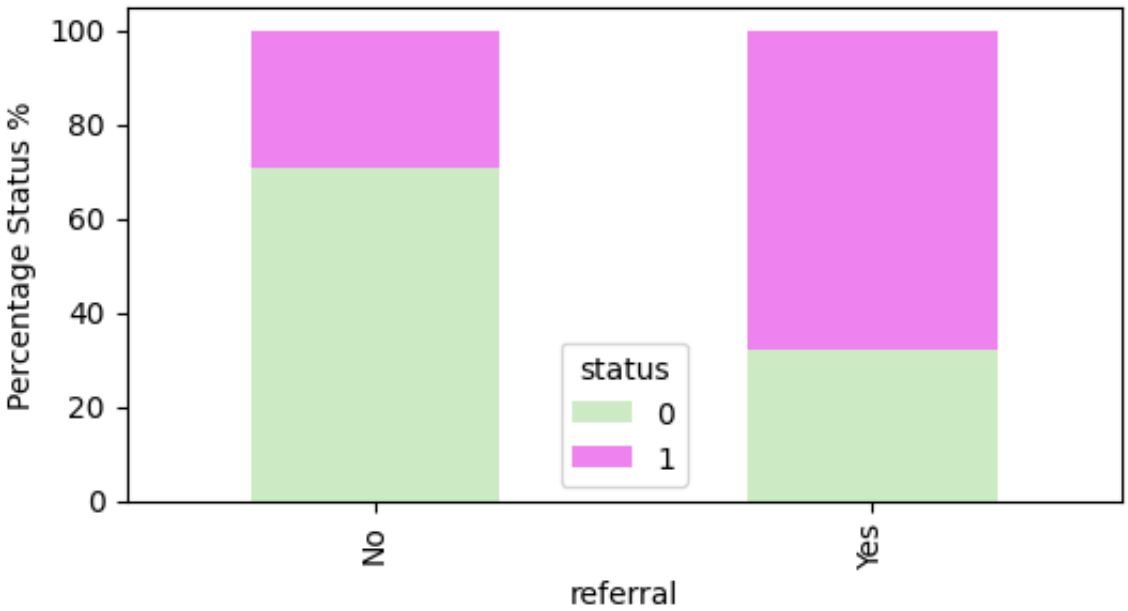

status
0
1
Percentage Status %
100
80
60
40
20
0
Email Activity
Phone Activity
Website Activity
last_activity


100
80
60
40
20
0
Percentage Status %
status
0
1
No
Yes
print_media_type1


100
80
60
40
20
0
Percentage Status %
status
0
1
No
Yes
print_media_type2


100
80
60
40
20
0
Percentage Status %
status
0
1
No
Yes
digital_media


100
80
60
40
20
0
Percentage Status %
status
0
1
No
Yes
educational_channels



In this group of crosstab plots we can see that convert more:

- Leads that are professionals
- Leads which their first_interaction through the website
- Leads with high or medium level of completion of their profiles online have better probabilities of being converted to paid customers. This variables are analyzed separately one from the other, so these are not final conclussions. The rest of the variables seem not to have a clear effect in the convertion chances.

Categorical variables with counts per Status

In [118…

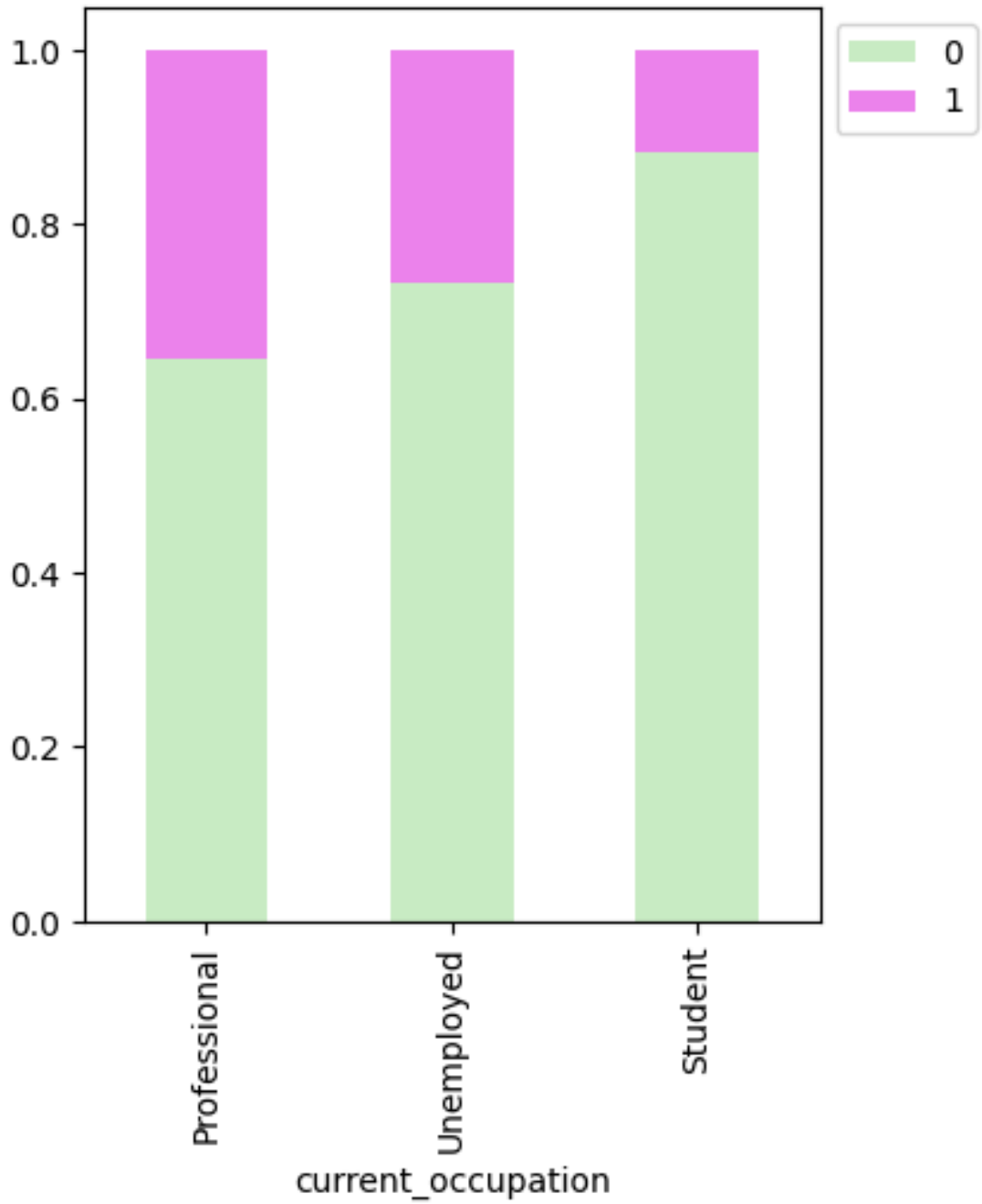
```
# Function to plot stacked bar plots

def stacked_barplot(data, predictor, target):
    """
    Print the category counts and plot a stacked bar chart

    data: dataframe
    predictor: independent variable
    target: target variable
    """
    count = data[predictor].nunique()
```

```
    sorter = data[target].value_counts().index[-1]
    tab1 = pd.crosstab(data[predictor], data[target], margir
        by = sorter, ascending = False
    )
    print(tab1)
    print("-" * 120)
    tab = pd.crosstab(data[predictor], data[target], normali
        by = sorter, ascending = False
    )
    pastel_colors = ['#ccebc5','violet']
    tab.plot(kind = "bar", stacked = True, figsize = (count
             color=['#ccebc5','violet'])
    plt.legend(
        loc = "lower left",
        frameon = False,
    )
    plt.legend(loc = "upper left", bbox_to_anchor = (1, 1))
    plt.show()
```

In [119…

```
# Visualize several Stacked Barplots for categorical variabl
#validate conclussions observed in the previous graphs
stacked_barplot(data, "current_occupation", "status")
```

```
status                 0     1   All
current_occupation
All                 3223  1375  4598
Professional        1681   927  2608
Unemployed          1053   383  1436
Student              489    65   554
------------------------------------------------------------
------------------------------------------------------------
```

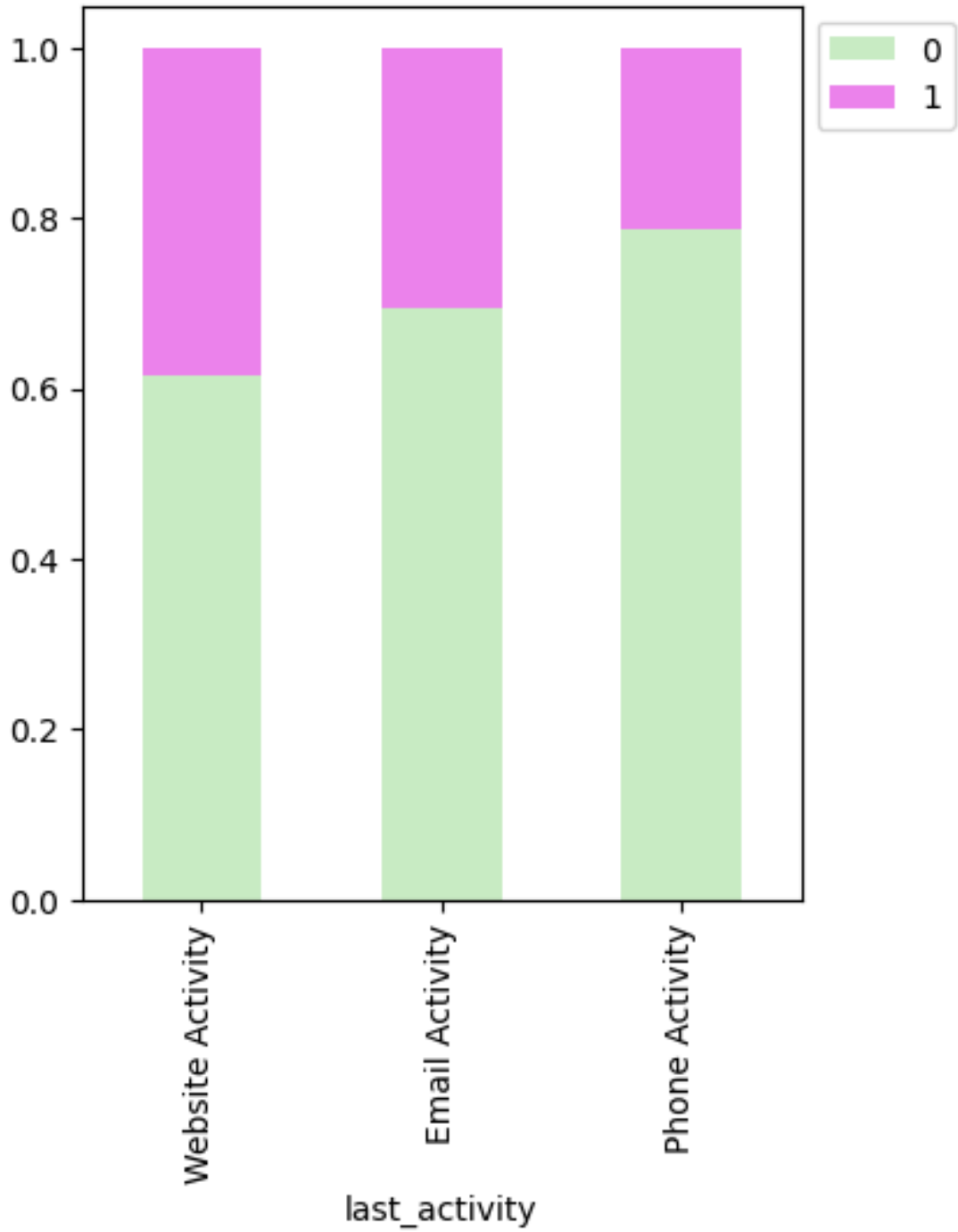


In quantity 927 (67%) of the total of leads that became paid customers (1,375), are professionals. Then 383 are unemployed and just 65 are students. In quantity the leads that are professional (2,608) are way higher than the unemployed (1,436) and the students (554), therefore professionals would have more chances to convert since the leads are more in quantity.

In [120…

```
stacked_barplot(data, "last_activity", "status")
color=['#ccebc5','violet']
```

```
status                0     1   All
last_activity
All                3223  1375  4598
Email Activity     1579   690  2269
Website Activity    677   423  1100
Phone Activity      967   262  1229
-------------------------------------------------------------
-------------------------------------------------------------
```



Within the internal analysis of last_activity the website activity have better convertion rate with almost 39%

- Email Activity: 690/2269 = 30.4%
- Website Activity:423/1100 = 38.5%
- Phone Activity: 262/1229 = 21.3%

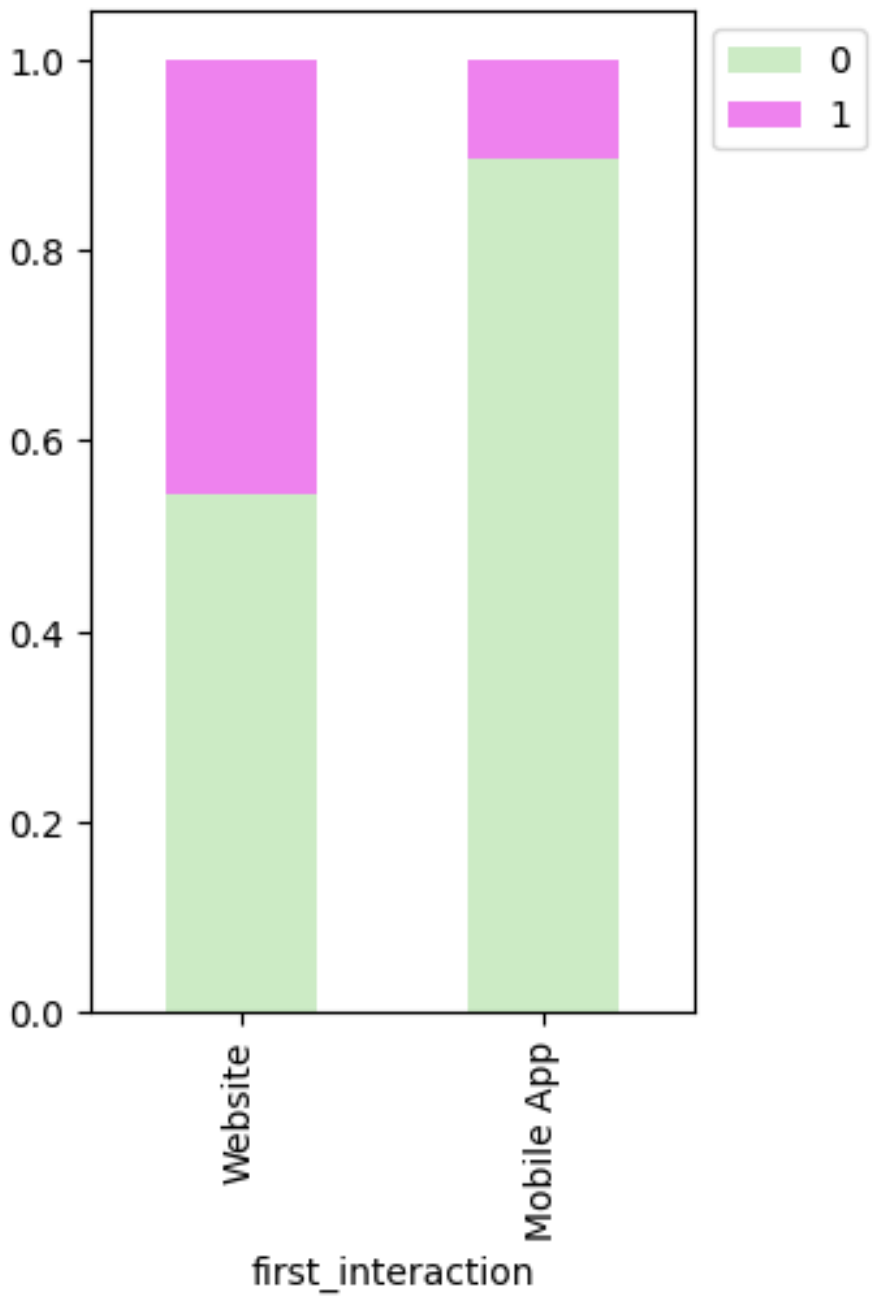
But in absolute terms the last_activity that predict more paid leads is Email Activity. I would think that this is because the lead is already in the last phase, right before the convertion to paid lead, or in fact is already enrolled.

- Email Activity: 690
- Website Activity: 423
- Phone Activity: 262

In [121…

```
stacked_barplot(data, "first_interaction", "status")
color=['#ccebc5','violet']
```

```
status                0     1   All
first_interaction
All                3223  1375  4598
Website            1379  1157  2536
Mobile App         1844   218  2062
------------------------------------------------------------
------------------------------------------------------------
```

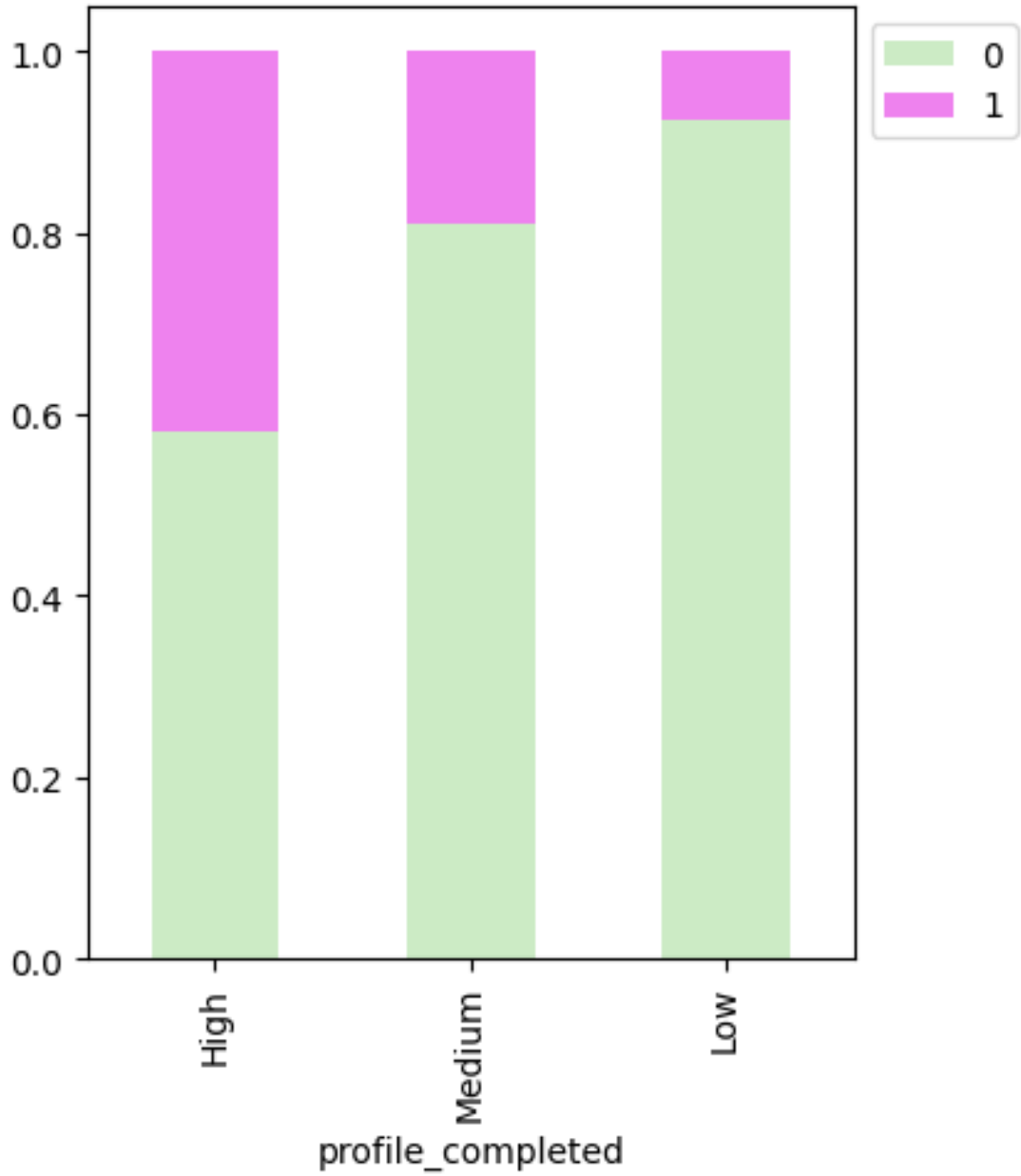


In quantity 1,157 (84%) of the total of leads that became paid customers (1,375), had their first_interaction by website, the rest (218) aprox. 16% had their fisrt_interaction by the mobile app. When we just analyze the category, the website has 1,157/ 2,536 positive cases (46%), in comparison with the mobile_app that just a little 218/2,062 positives (~11%). In both scenarios, in absolute terms and by category, the website first interaction has more changes to convert to a paid customer.

In [122…

```
stacked_barplot(data, "profile_completed", "status")
color=['#ccebc5','violet']
```

```
status                 0     1   All
profile_completed
All                 3223  1375  4598
High                1311   944  2255
Medium              1813   423  2236
Low                   99     8   107
-------------------------------------------------------------
-------------------------------------------------------------
```

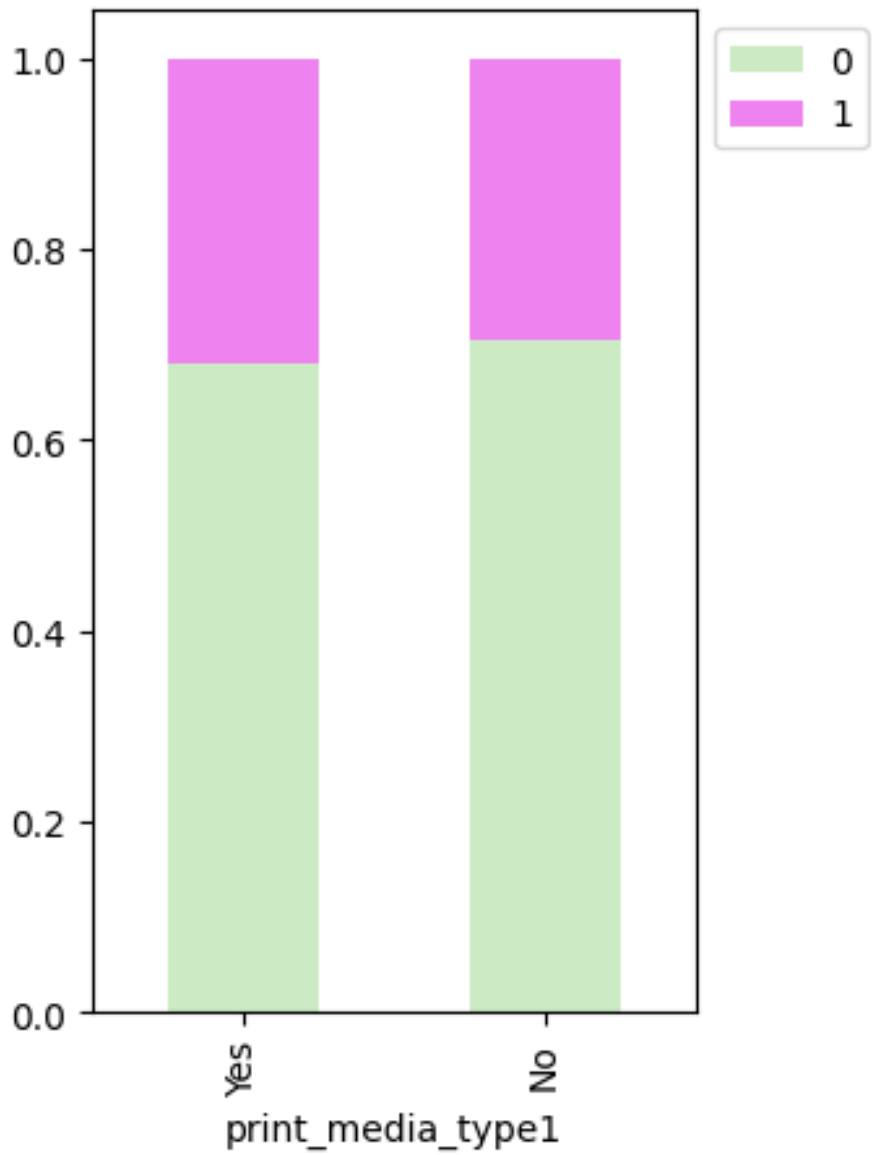


In quantity 944 (~69%) of the total of leads that became paid customers (1,375), have high level of their profiled completed. This variable has good chances to predict the convertion to paid customers, the quantity of leads are high in total, high level has 2255 leads and medium level has 2236 leads, with a total between them of 4491 out of the 4598 ~ 98% profile_completed, and the internal convertion rate is high as well per category, having high profile completion 944/255 ~ 42% and medium profile completion 423/2236 ~ 19%, which is not despreciable.

In [123…

```
stacked_barplot(data, "print_media_type1", "status")
color=['#ccebc5','violet']
```

```
status                 0     1   All
print_media_type1
All                 3223  1375  4598
No                  2886  1216  4102
Yes                  337   159   496
------------------------------------------------------------
------------------------------------------------------------
```

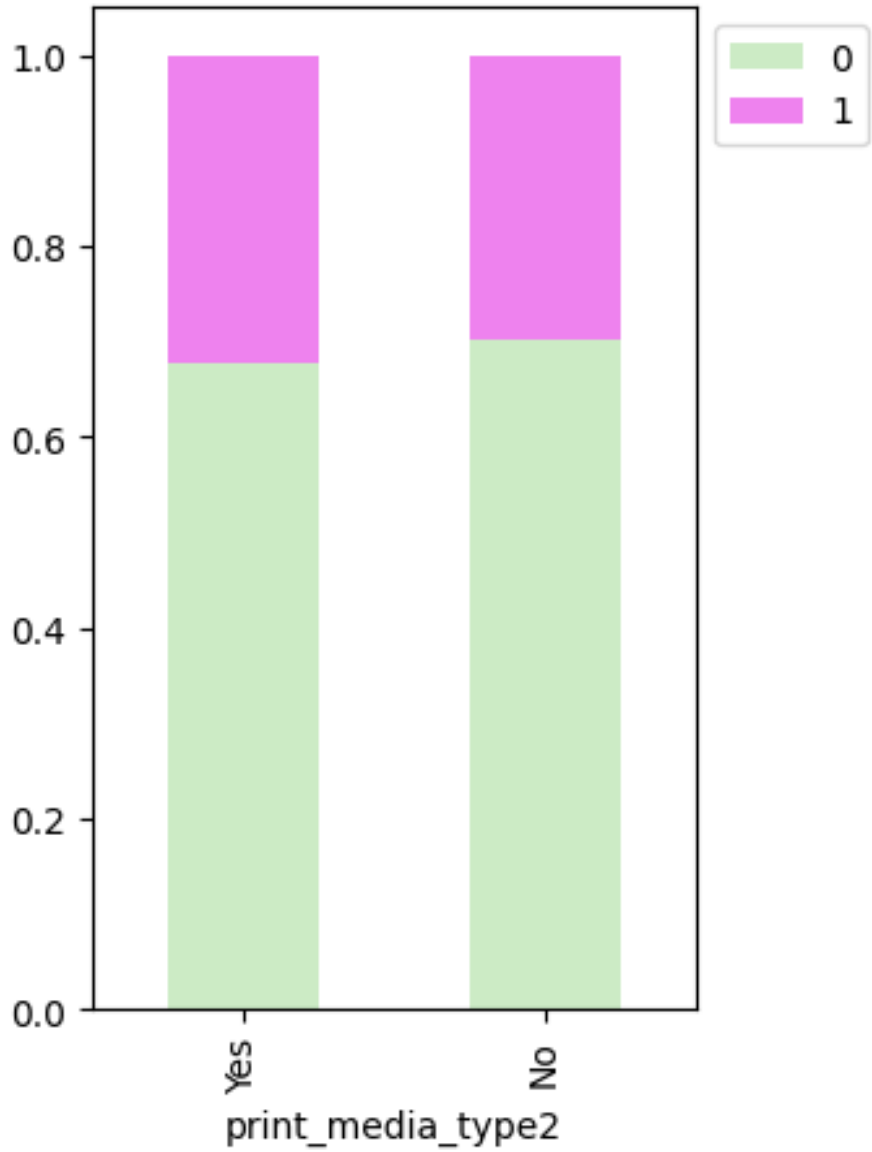


In quantity 159 (~11%) of the total of leads that became paid customers (1,375), came from newspaper adds. This variable has very small chances to convert to a paid customers the leads are very few in total 496 out of 4598 ~ 11%, and the internal convertion rate is a little bit low as well 159/496 ~ 32%.

In [124…

```
stacked_barplot(data, "print_media_type2", "status")
color=['#ccebc5','violet']
```

```
status                 0     1   All
print_media_type2
All                 3223  1375  4598
No                  3065  1300  4365
Yes                  158    75   233
-------------------------------------------------------------
-------------------------------------------------------------
```

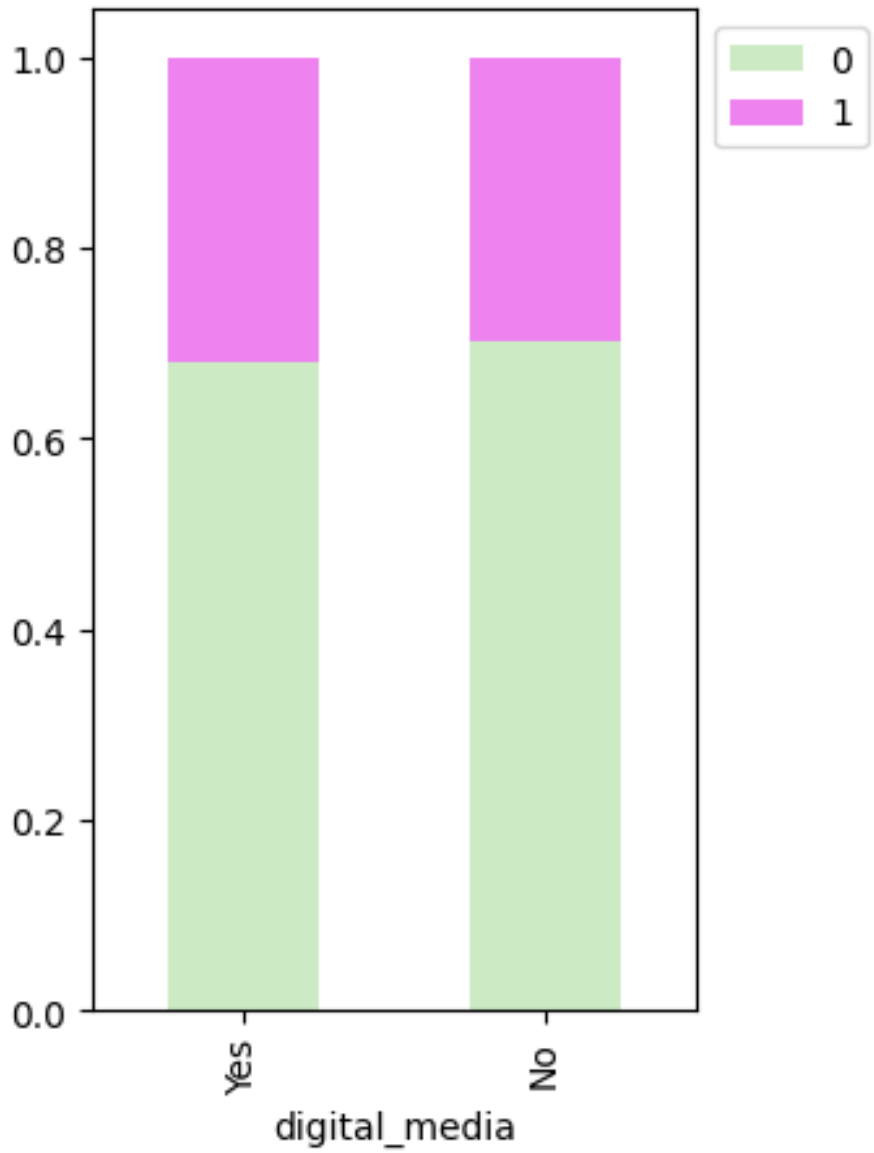


In quantity 75 (~5%) of the total of leads that became paid customers (1,375), came from magazine adds. This variable has very small chances to convert to a paid customers, the leads are very few in total 233 out of 4598 ~ 5%, and the internal convertion rate is a little bit low as well 75/233 ~32%.

In [125…

```
stacked_barplot(data, "digital_media", "status")
color=['#ccebc5','violet']
```

```
status            0     1   All
digital_media
All            3223  1375  4598
No             2864  1207  4071
Yes             359   168   527
-------------------------------------------------------------
-------------------------------------------------------------
```



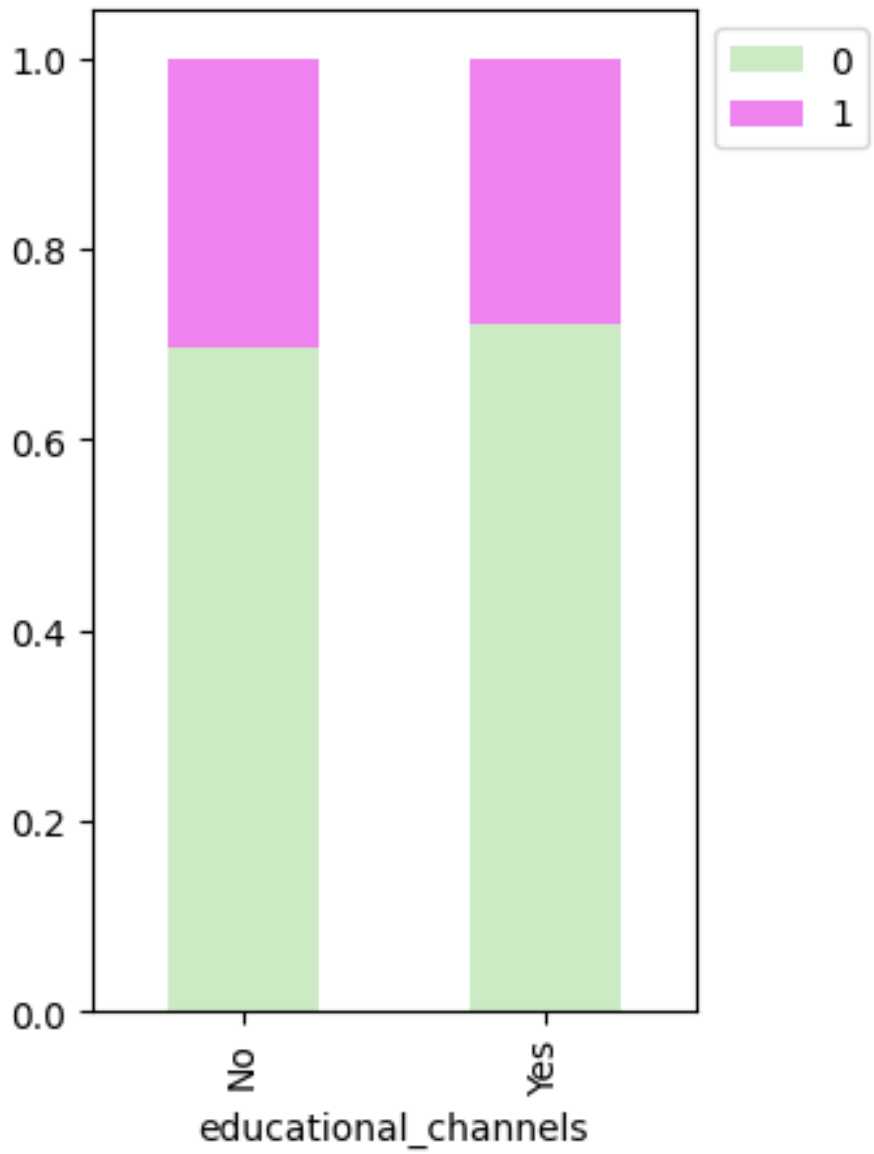
In quantity 168 (~12%) of the total of leads that became paid customers (1,375), came from digital_media. This variable has very small chances to convert to a paid customers, the leads are very

few in total 527 out of 4598 ~ 11%, and the internal convertion rate is a little bit low as well 168/527 ~ 32%.

In [126…

```
stacked_barplot(data, "educational_channels", "status")
color=['#ccebc5','violet']
```

```
status                    0     1   All
educational_channels
All                    3223  1375  4598
No                     2716  1178  3894
Yes                     507   197   704
------------------------------------------------------------
------------------------------------------------------------
```

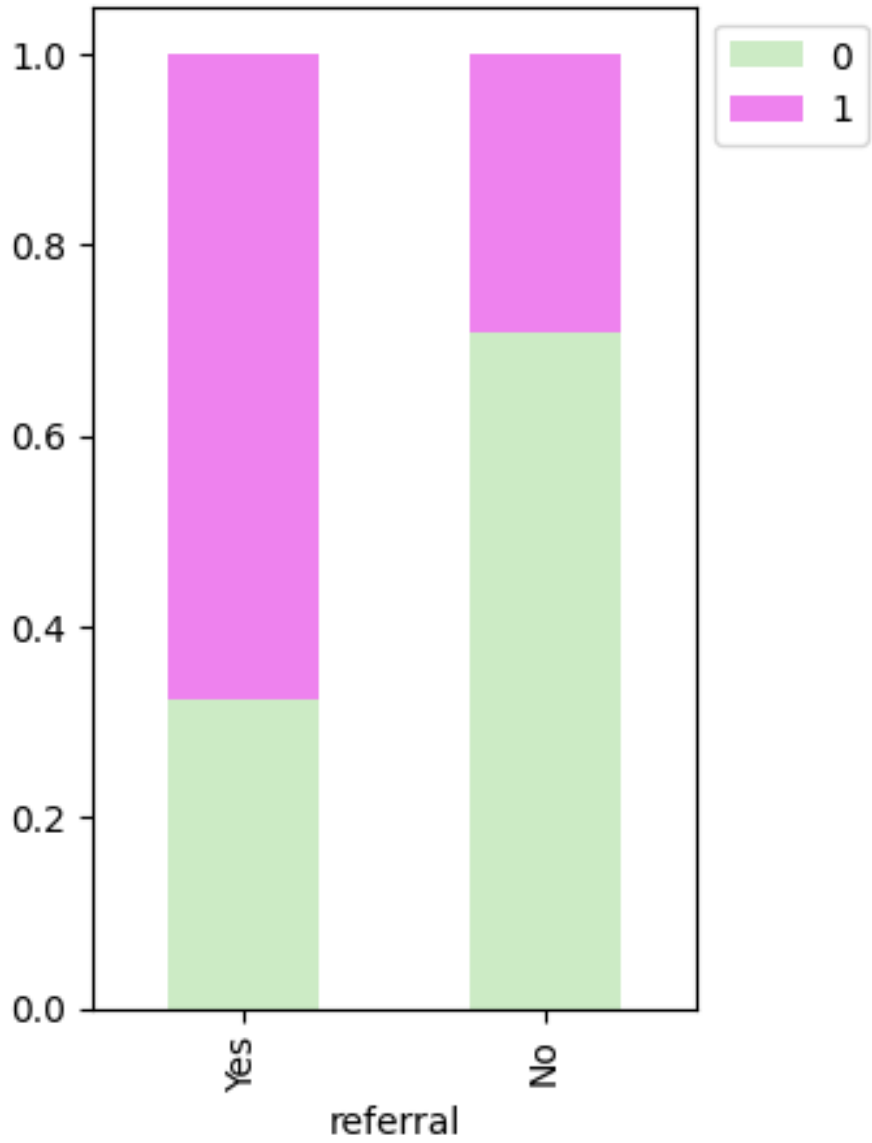


In quantity 197 (~14%) of the total of leads that became paid customers (1,375), came from educational_channels. This variable has better chances to convert to a paid customers than the previous ones, the leads that come from this variable are in total 704 out of 4598 ~ 15%, but the internal convertion rate is a little bit lower than the previous variables with 197/704 ~ 28%.

In [127…

```
stacked_barplot(data, "referral", "status")
color=['#ccebc5','violet']
```

```
status       0     1   All
referral
All       3223  1375  4598
No        3193  1312  4505
Yes         30    63    93
------------------------------------------------------------
------------------------------------------------------------
```



In quantity only 63 (4%) of the total of leads that became paid customers (1,375), came from referrals. Eventhough, 63 out of the 93 total that were referred, is a good convertion rate (~68%). This should be highlighted as a **marketing campaign to increase the quantity of referrals since this variable has a high chance of convertion**

***Observations for Crosstab plots**: The following observations and responses are extracted from the previous set of crosstab plots

- Response to question 1: ***The current occupation affect lead status. It shows that more than 35% of the professionals and more than 25% of the leads that are unemployed decided to participate in the program***. Very little portion, on the other hand, less than 10% of the group that are already studying, decide to join the program. Professionals may want to be always with updated skills or are enrolled by their employees, and the unemployed try to reskill to get better job opportunities.

- Response to question 2: Yes, the company first impression on the customer have definetely an impact. **More than 40% of the leads that interact through the website join the program, but just a ~10% of the lead that interact via mobile app convert to paid customers**. Maybe the website provides a better view of the available programs and easier interaction than the mobile app.

- Response to question 3: This is an interesting conclussion, even when in a previous graph we saw the the email way of interaction is the most common in absolute terms, ***the website interaction with ~38% is the one that work best to convert to paid customers more than the email and the phone interactions***. The phone interaction has the smaller chances to convert the leads to paid customers.

- Response to question 4: ***The channel with the highest lead convertion rate ~68% is referral in terms of effectiveness*** since none of the other channels like print media, digital media, educational channels have better convertion rates even when they have higher quantity of leads. If we just see the row numbers the channel with the highest quantity of leads that became paid customer is educational with 197, followed by digital_media with 168 leads, but in the end, all

of the channels represent very small portions to blindly be considered as important variables for the convertion. Even when we have very few leads per channel, given the fact that there are always multiple sources involved, each of channels have an approximate of 5-10% as portion of the whole marketing source of leads, and it is interesting that internally the convertion per category goes around the 32%, which means that once the lead is in the pipeline, depending of the ability and tactics utilized, the leads have a fair chance to become paid customers.

- Response to question 5: ***Having more details about the prospect definitely increase the chances of convertion to paid customer***. The least we know about the lead, the least oppotunities we have to convert. High level of profile completion represent 69% of the total of paid customers. As I previously explained before, this variable has good chances to predict the convertion to paid customers, since there are 2 positive factors, the quantity of leads that have completed their profiles in high/medium level are high in total, 4491 out of the 4598 ~ 98%, and the internal convertion rate is high as well per category for the high profile completion 944/255 ~ 42%.

Data Preprocessing

- Missing value treatment was not neccesary
- Feature engineering
- Outlier detection and treatment: Decision trees are good at handling outlier, so I am not going to treat for outliers
- Preparing data for modeling
- Any other preprocessing steps

Feature engineering

In [128…

```
#Create new column
data['avg_time_per_visit'] = data['time_spent_on_website']/c
```

In [129…

```
#Verify the last column created avg_time_per_visit at the en
data.head ()
```

Out[129…

	age	current_occupation	first_interaction	profile_completed	w
0	57	Unemployed	Website	High	
1	56	Professional	Mobile App	Medium	
2	52	Professional	Website	Medium	
3	53	Unemployed	Website	High	
4	23	Student	Website	High	

In [130…

```
#Average time per visit value validation
data['avg_time_per_visit'].describe()
```

Out[130…

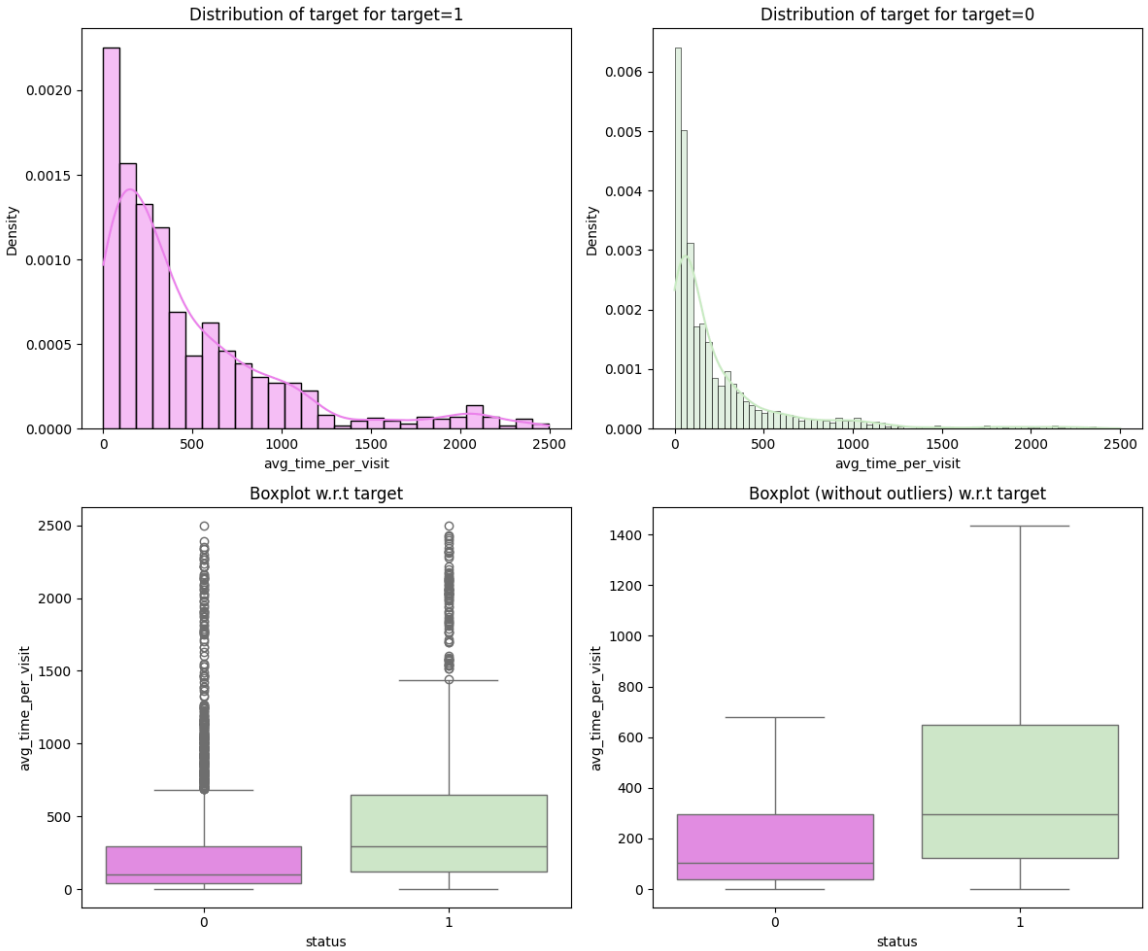
	avg_time_per_visit
count	4438.000000
mean	315.988926
std	426.465913
min	0.066667
25%	49.678571
50%	151.750000
75%	390.712500
max	2500.000000

dtype: float64

There are minimum at 0 and max at 2500 in average time per visit.

In [131…

```
#Plot Wrt Target to visualize new variable
distribution_plot_wrt_target(data, "avg_time_per_visit", "st
```



If we do not take the outliers in consideration, in general we can see that the leads that spent more than 300 seconds/5 minutes in the website in average, are the leads that tend to convert and enroll in the program. This new ***specific factor have lots of sense since the leads that are really interested are going to spend more time in researching about details of the programs***.

In [132…

```
# Binning the age column to get a range of age instead of sp
data['age'] = original_data['age'].copy() # Reset 'age' to c
bins = [18, 30, 40, 50, 60, np.inf]
labels = ['18-30', '31-40', '41-50', '51-60', '61+']
data['age'] = pd.cut(data['age'], bins=bins, labels=labels,
```

Preparing data for modeling

In [133…

```
# ONE HOT ENCODE/Creating dummy variables for the categorica
# drop_first=True is used to avoid redundant variables
data = pd.get_dummies(
```

```
    data,
    columns = data.select_dtypes(include = ["object", "categ
    drop_first = True,
)
```

In [134…

```
#Visualize the new dummy columns created
data.head()
```

Out[134…

	website_visits	time_spent_on_website	page_views_per_visit	sta
0	7	1639	1.861	
1	2	83	0.320	
2	3	330	0.074	
3	4	464	2.057	
4	4	600	16.914	

The number of columns has increased after the creation of the dummy variables. For each categorical column the first category has been removed to be the reference variable.

In [135…

```
# Separating independent variables and the target variable s
x = data.drop('status',axis=1)
y = data['status']

# Ensure  that no NaNs or Infs in x before splitting
x.replace([np.inf, -np.inf], np.nan, inplace=True)
x.fillna(0, inplace=True)

# Print NaNs or Infs Infinites if present in x
print("Number of NaN values in x before train-test split:",
print("Number of Inf values in x before train-test split:",
```

```
Number of NaN values in x before train-test split: 0
Number of Inf values in x before train-test split: 0
```

In [136…

```
# Splitting the dataset into train and test datasets
x_train, x_test, y_train, y_test = train_test_split(x, y, te
```

In [137…

```
# Checking the shape of the train and test data
print("Shape of Training set : ", x_train.shape)
print("Shape of test set : ", x_test.shape)
```

```
Shape of Training set :  (3218, 20)
Shape of test set :  (1380, 20)
```

In [138…

```
# Importing again classification metrics to measure the perf
from sklearn.metrics import accuracy_score, precision_score,

# Function to compute different metrics to check performance
def model_performance_classification(model, predictors, targ
    """
    Function to compute different metrics to check classific
    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    pred = model.predict(predictors)                    # Prec
    prob = model.predict_proba(predictors)[:, 1]        # Get

    accuracy = accuracy_score(target, pred)             # To c
    precision = precision_score(target, pred)           # To c
    recall = recall_score(target, pred)                 # To c
    f1 = f1_score(target, pred)                         # To c
    roc_auc = roc_auc_score(target, prob)               # To c

    # Creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {
            "Accuracy": accuracy,
            "Precision": precision,
            "Recall": recall,
            "F1-score": f1,
            "ROC AUC": roc_auc,
        },
        index=[0],
    )
```

```
    return df_perf
```

Building a Decision Tree model

Decision Tree Classifier - Train and Test

In [139…

```
# Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state = 1)

# Fitting the model (ensuring it's fitted on the most recent
dt_classifier.fit(x_train, y_train)

# Model Performance on the training data
dt_classifier_perf_train = model_performance_classification(
dt_classifier_perf_train
```

Out[139…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.998757	1.0	0.995897	0.997945	0.999996

First run: For the train performance my accurancy 0.99, precision 1, recall 0.99, F1 0.99 and ROC AUC 0.99, given the fact that all my scores are very high, it seems my train data is memorized by the model and it will be difficult to work well in the test data. We will see.

In [140…

```
# DT Model Performance on the test data
dt_classifier_perf_test = model_performance_classification(c
dt_classifier_perf_test
```

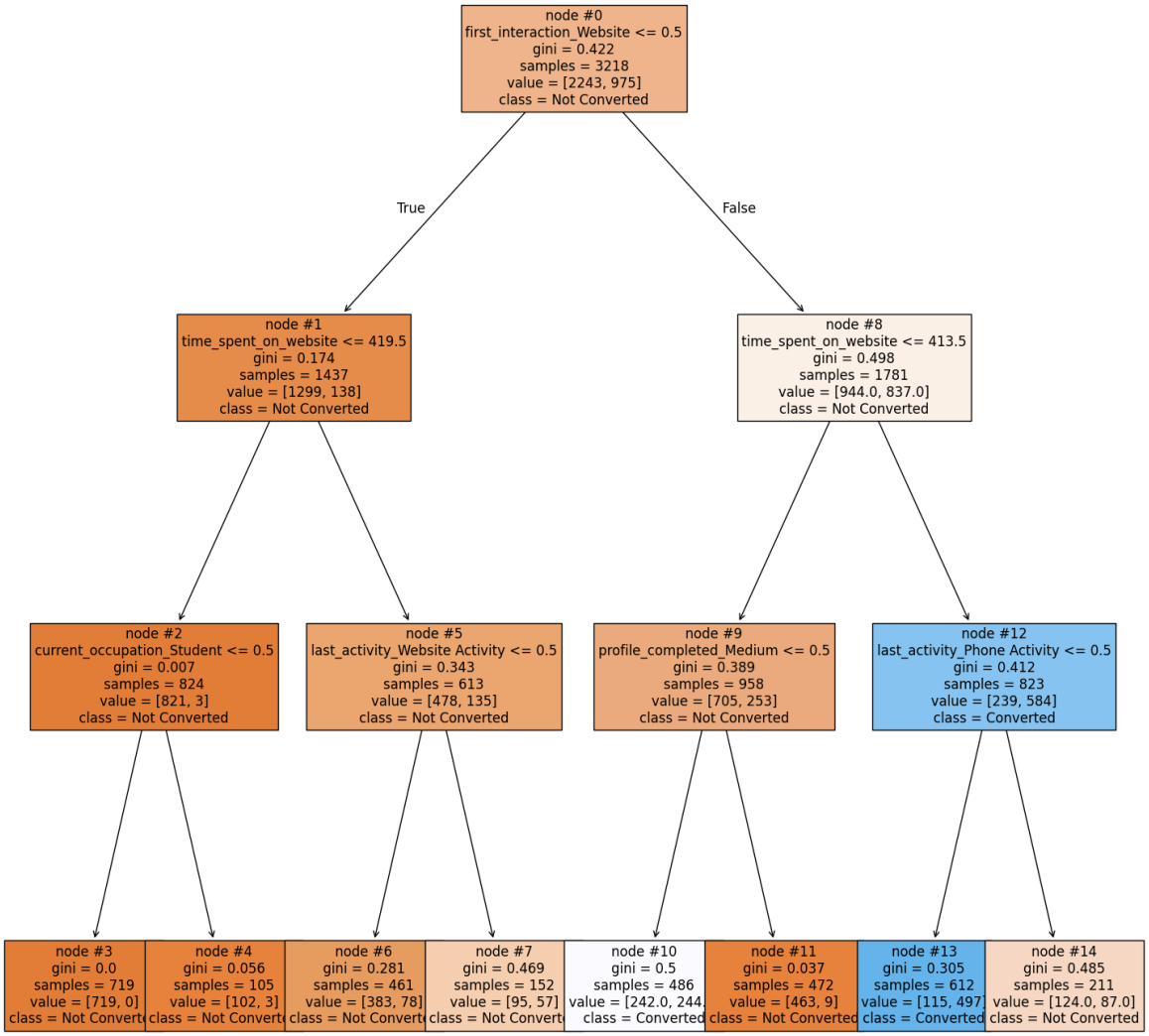
Out[140…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.797101	0.638889	0.69	0.663462	0.770347

First run: Exactly as I thought, for my test performance my accurancy 0.79, precision 0.64 , recall 0.69, F1 0.66 and ROC AUC 0.77 are low, very far from my scores in the trainning data. My model is overfitted and it is not capturing the real structure of the problem. Maybe because I did not correct outliers it is capturing noise. I will have to do something to get a better model.

In [141…

```
# Limit the decision tree's depth so that we can visualize i
from sklearn import tree
features = list(x.columns)

# Building the model with max_depth=3 and max_depth=4 to dec
dt_classifier_visualize = DecisionTreeClassifier(random_stat

# Fitting the model
dt_classifier_visualize.fit(x_train, y_train)


plt.figure(figsize = (18, 20))
tree.plot_tree(dt_classifier_visualize, feature_names = feat
               node_ids = True, class_names = ['Not Converte
plt.show()
```



I tried to plot with max depth 4 and the decision tree gets more difficult to read, and gives other few variables than with the depth 3, but I decided to go back and let the graph with depth 3. The white and blue rectangules converted. The rest oranges do not convert.

In [142…

```
# Printing the Decision Tree content
print(tree.export_text(dt_classifier_visualize, feature_name
```

```
|--- first_interaction_Website <= 0.50
|   |--- time_spent_on_website <= 419.50
|   |   |--- current_occupation_Student <= 0.50
|   |   |   |--- weights: [719.00, 0.00] class: Not Converted
|   |   |--- current_occupation_Student >  0.50
|   |   |   |--- weights: [102.00, 3.00] class: Not Converted
|   |--- time_spent_on_website >  419.50
|   |   |--- last_activity_Website Activity <= 0.50
|   |   |   |--- weights: [383.00, 78.00] class: Not Converte
d
|   |   |--- last_activity_Website Activity >  0.50
|   |   |   |--- weights: [95.00, 57.00] class: Not Converted
|--- first_interaction_Website >  0.50
|   |--- time_spent_on_website <= 413.50
|   |   |--- profile_completed_Medium <= 0.50
|   |   |   |--- weights: [242.00, 244.00] class: Converted
|   |   |--- profile_completed_Medium >  0.50
|   |   |   |--- weights: [463.00, 9.00] class: Not Converted
|   |--- time_spent_on_website >  413.50
|   |   |--- last_activity_Phone Activity <= 0.50
|   |   |   |--- weights: [115.00, 497.00] class: Converted
|   |   |--- last_activity_Phone Activity >  0.50
|   |   |   |--- weights: [124.00, 87.00] class: Not Converte
d
```

Observations:

- **Root Node:** First_interaction_website <= 0.5. This is the starting point of the decision tree, which means that the First_interaction_website results in the highest information gain among all the features. If the value is less than or equal to 0.5, the left branch is taken, and if it is greater than 0.5, the right branch is taken.

- **Internal Nodes:**

  - Time_spent_website <= 419.5
  - Time_spent_website <= 413.5
  - Current Ocupation/Student <= 0.5

- Last_activity/Website <= 0.5
- Profile_completed/Medium <= 0.5
- Last_activity/Phone <= 0.5

Interpretation and Conclusions:

The decision tree starts with the variable first_interaction_Website > 0.50, with a split on the Time_spent_website feature.

- If the lead first_interaction_Website > 0.50, and then the time_spent_on_website <= 413.50, and the profile_completed_Medium <= 0.50, the lead convert to paid customer = Total samples 486

- The second path if the lead first_interaction_Website > 0.50, and then the time_spent_on_website > 413.50 and the last_activity_Phone Activity <= 0.50, the lead convert to paid customer = Total samples 612

Note: My tree is truncated and not shown completely to get proper visualization.

Building and evaluating other models

Bagging Classifier

In [143…

```
# Bagging Classifier
bagging_estimator = BaggingClassifier(random_state = 1)

# Fitting the model
bagging_estimator.fit(x_train, y_train)

# Model Performance on the train data
bagging_estimator_perf_train = model_performance_classificat
bagging_estimator_perf_train
```

Out[143…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.987881	0.988518	0.971282	0.979824	0.999109

First run: It happened the same, my train metrics are very high in general above 0.97.

In [144…

```
# Model Performance on the test data
bagging_estimator_perf_test = model_performance_classificati
bagging_estimator_perf_test
```

Out[144…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.839855	0.745205	0.68	0.711111	0.881677

First run: The train metrics are low, the lowest is the recall with 0.68. Very distance from the train metrics, so overfitted.

Random Forest Classifier

In [145…

```
# Random Forest Classifier is the model selected after runni
rf_classifier = RandomForestClassifier(n_estimators = 100, r

# Fitting the model
rf_classifier.fit(x_train, y_train)

# Model Performance on the train data
rf_classifier_perf_train = model_performance_classification(
rf_classifier_perf_train
```

Out[145…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.998757	0.997949	0.997949	0.997949	0.999996

First run: I see the same pattern for this model as well, the train metrics are very very high, which is suspicious.

In [146…

```
# Model Performance on the test data
rf_classifier_perf_test = model_performance_classification(r
rf_classifier_perf_test
```

Out[146…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.843478	0.747312	0.695	0.720207	0.902856

First run: For the test performance my accurancy 0.84, precision 0.74, recall 0.69, F1 0.72 and ROC AUC 0.90 are a far in general from my RF train test performance. My RF model is overfitted as well and this problem needs to be corrected since my precision and recall are very distance from train and test, and specially if I am interested in reducing the false negative / RECALL rate, to avoid Extralearn loose potencial paid customers. This problem might be because my data is unbalance the class 0 is bigger than the class 1. I need to level it.

Exercise RF CLASSIFIER 2: Adjusting manually the parameters for the Random Forest and Leveling data weight

In [147…

```
# RF Model Performance on the train data limiting the comple
# adjusting number of estimators, balancing the classes, lim
# increasing min_sample_leaf,
rf_classifier2 = RandomForestClassifier(n_estimators=300, ma
                                        min_samples_split=50
                                        min_samples_leaf=10,
                                        class_weight='balanc
                                        max_features='sqrt',
                                        random_state=42)

# Fitting the model RF2
rf_classifier2.fit(x_train, y_train)
```

```
# Model Performance 2 on the train data
rf_classifier_perf_train2 = model_performance_classificatior
rf_classifier_perf_train2
```

Out[147…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.864201	0.723794	0.892308	0.799265	0.949002

First run: The firt time I adjusted the RF, I just did the data weight balance, but it was not enough, since the all the train metrics remained very high from 0.99 to 1.

Second run: After adjusting n_estimators=300, max_depth=12, min_samples_split=50, min_samples_leaf=10, class_weight='balanced', max_features='sqrt', random_state=42, I got better precision 0.72, recall 0.89 and F1 0.79 performance metrics for the train dataset.

In [148…

```
# Model Performance on the test data after leveling classes
rf_classifier_perf_test2 = model_performance_classification(
rf_classifier_perf_test2
```

Out[148…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.823913	0.654832	0.83	0.732084	0.90637

First run: By just balancing classes, I could not improve that much, the test metrics remain very low and far from the train. Recall = 0.69 is the lowest, but precision and F1 are around the 0.72.

Second run: After adjusted several parameters mentioned in the train section, the model was able to improve getting better test performance metrics, more near to the precision, recall and F1 score from the train. ***Recall improved significantly in test from the first run 0.69, to 0.83 in this second run helping to reduce***

the false negatives, but Precision, as the trade-off work, decreased from 0.72 to 0.65 which means now my false positive would increase. Anyways this second run after adjusting parameters, is way better than the first run since we were able to overcome the overfitting.

ADA Boost Classifier

In [149…

```
# AdaBoost Classifier
ada_classifier = AdaBoostClassifier(random_state=1)

# Fitting the model
ada_classifier.fit(x_train, y_train)

# Model Performance on the train data
ada_classifier_perf_train = model_performance_classificatior
ada_classifier_perf_train
```

Out[149…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.831572	0.769614	0.633846	0.695163	0.903906

First run: Accuracy 0.83, precision 0.76, Recall 0.63, F1 0.69 and ROC AUC 0.90. ADA classifier gave better metrics in comparison with the previous models, but low still to be acceptable for the case purposes.

In [150…

```
# Model Performance on the test data
ada_classifier_perf_test = model_performance_classification(
ada_classifier_perf_test
```

Out[150…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.807971	0.703927	0.5825	0.637483	0.869128

First run: Accuracy 0,80, precision 0,70, Recall 0.58, F1 0.63 and ROC AUC 0.87. Recall is too low, very far to be a good model to predict paid customers. At least this model is not overfitted.

Gradient Boosting Classifier

In [151…

```
# Gradient Boosting Classifier
grad_classifier = GradientBoostingClassifier(random_state=1)

# Fitting the model
grad_classifier.fit(x_train, y_train)

# Model Performance on the train data
grad_classifier_perf_train = model_performance_classificatic
grad_classifier_perf_train
```

Out[151…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.887508	0.834242	0.784615	0.808668	0.950634

First run: Accuracy 0.88, precision 0.83, Recall 0.78, F1 0.80 and ROC AUC 0.95. This results are the best until the moment for the train set, better than in the previous models because are not near to 1 memorizing the train set and overfiting in the end, like in the DT and RF fisrt runs.

In [152…

```
# Model Performance on the test data
grad_classifier_perf_test = model_performance_classificatior
grad_classifier_perf_test
```

Out[152…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.844203	0.74026	0.7125	0.726115	0.908446

First run: Accuracy 0,84, precision 0,74, Recall 0.71, F1 0.73 and ROC AUC 0.90. Better recall than in the previous models, not that

low, and when compared with the train metrics recall 0.78 is not that far. Not overfitted model

XGBoost Classifier

In [153…

```
# Installing the xgboost library using the 'pip' command
# !pip install xgboost
```

In [154…

```
# Importing XGBoost Classifier
from xgboost import XGBClassifier

# XGBoost Classifier with added use_label_encoder and eval_m
xgb = XGBClassifier(random_state = 1, use_label_encoder=Fals

# Fitting the model
xgb.fit(x_train,y_train)

# Model Performance on the train data
xgb_perf_train = model_performance_classification(xgb, x_tra
xgb_perf_train
```

Out[154…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.997203	0.991853	0.998974	0.995401	0.999778

First run: Accuracy 0.99, precision 0.99, Recall 0.99, F1 0.99 and ROC AUC 0.99. All very high, following the same pattern than previous models DT and RF.

In [155…

```
# Model Performance on the test data
xgb_perf_test = model_performance_classification(xgb, x_test
xgb_perf_test
```

Out[155…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.830435	0.715026	0.69	0.70229	0.898661

First run: Accuracy 0,83, precision 0,71, Recall 0.69, F1 0.70 and ROC AUC 0.89.

Model Performance evaluation and improvement

In [156…

```
models_test_comp_df = pd.concat(
    [
        dt_classifier_perf_test.T,
        bagging_estimator_perf_test.T,
        rf_classifier_perf_test.T,
        rf_classifier_perf_test2.T,
        ada_classifier_perf_test.T,
        grad_classifier_perf_test.T,
        xgb_perf_test.T
    ],
    axis = 1,
)

models_test_comp_df.columns = [
    "Decision Tree Classifier",
    "Bagging Classifier",
    "Random Forest Classifier",
    "Random Forest Classifier 2",
    "Ada Boost Classifier",
    "Gradient Boosting Classifier",
    "XG Boost Classifier"]

print("Test performance comparison:")

models_test_comp_df.T
```

```
Test performance comparison:
```

Out[156…

	Accuracy	Precision	Recall	F1-score	ROC AUC
Decision Tree Classifier	0.797101	0.638889	0.6900	0.663462	0.770347
Bagging Classifier	0.839855	0.745205	0.6800	0.711111	0.881677
Random Forest Classifier	0.843478	0.747312	0.6950	0.720207	0.902856
Random Forest Classifier 2	0.823913	0.654832	0.8300	0.732084	0.906370
Ada Boost Classifier	0.807971	0.703927	0.5825	0.637483	0.869128
Gradient Boosting Classifier	0.844203	0.740260	0.7125	0.726115	0.908446
XG Boost Classifier	0.830435	0.715026	0.6900	0.702290	0.898661

First run: Between all the models that I run, the Gradient Boosting Classifier worked better. **The test recall was 0.71 for the first run, with the test sample 0.30 and without leveling the data.**

Second run: ***After I adjusted parameters for the Random Forest Classifier manually following best practices to reduce the complexity of the model in order to avoid the overfitting, I got Random Forest Classifier 2 with Recall 0.83 primarily focusing in that missing a positive customer is very costly. Anyways as we do not know the cost involved in this business, I prefer to mantain the F1 as the metric to take decision of which is the best model, and for this case, before tuning with Gridsearch, it is the Random

Forest Classifier 2, and in second place the Gradient Boosting model with recall 0.71**.

As requested in the rubric, I am going to follow the steps to tune the Decision Tree, again tune the Random Forest Classifier and the Gradient Boosting Classifier that was my first best model, and comment about the results after the model comparison table.

Tuning Decision Tree, Random Forest and Gradient Boosting Classifier

Tuned Decision Tree

In [157…

```
#GRID SEARCH METHOD FOR TUNING DT
dt_tuned = DecisionTreeClassifier(random_state = 1)

# Grid of parameters to choose from and entropy as recommenc
dt_parameters = {
    "max_depth": [5, 7, None],
    "min_samples_leaf": [1, 5, 10],
    "criterion": ["gini", "entropy"]
}

# Run the grid search with f1 to balance between recall and
dt_grid_obj = GridSearchCV(dt_tuned, dt_parameters, scoring

# Ensure x_train and y_train are defined by running previous
dt_grid_obj = dt_grid_obj.fit(x_train, y_train)

# Set the rf_tuned_classifier to the best combination of par
dt_tuned_classifier = dt_grid_obj.best_estimator_

dt_tuned_classifier.fit(x_train, y_train)

# Model Performance on the test data
dt_tuned_classifier_perf_test = model_performance_classifica

dt_tuned_classifier_perf_test
```

Out[157…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.837681	0.714634	0.7325	0.723457	0.88714

The model DT improved a little bit my recall after the tuning the DT.

Tuned Random Forest Classifier

In [158…

```
#GRID SEARCH METHOD FOR TUNING RF
rf_tuned = RandomForestClassifier(random_state = 1)

# Grid of parameters to choose from
rf_parameters = {"n_estimators": [100, 110, 120],

    "max_depth": [5, 7, None],

    "max_features": [0.8, 1]
             }

# Run the grid search with scoring f1 to balance
rf_grid_obj = GridSearchCV(rf_tuned, rf_parameters, scoring

rf_grid_obj = rf_grid_obj.fit(x_train, y_train)

# Set the rf_tuned_classifier to the best combination of par
rf_tuned_classifier = rf_grid_obj.best_estimator_

rf_tuned_classifier.fit(x_train, y_train)

# Model Performance on the test data
rf_tuned_classifier_perf_test = model_performance_classifica

rf_tuned_classifier_perf_test
```

Out[158…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.853623	0.761905	0.72	0.74036	0.9132

The recall is now 0.72 and precision 0.76 both of them have improved, and the F1 is 0.74, similar to the previous exercise when I tuned manually the Random Forest Classifier 2.

In [159…

```
#Using RandomizedSearchCV for this RFC


from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

## Define the RandomForestClassifier
rf_tuned = RandomForestClassifier(random_state=1)

# # Define the parameter distributions for randomized search
rf_parameters = {
     "n_estimators": randint(100, 121),  # Adjust the range
     "max_depth": [5, 7, None],
     "max_features": [0.8, 1]
 }

# # Run the randomized search
rf_random_search = RandomizedSearchCV(
     rf_tuned,
     param_distributions=rf_parameters,
     n_iter=10,  # Adjust the number of iterations as needed
     scoring='f1',
     cv=5,
     random_state=1,
     n_jobs=-1
 )

rf_random_search.fit(x_train, y_train)

# # Get the best estimator
rf_tuned_classifier = rf_random_search.best_estimator_

# # Fit the model with the best estimator
rf_tuned_classifier.fit(x_train, y_train)

# # Model performance on the test data
```

```
rf_tuned_classifier_perf_test = model_performance_classifica
rf_tuned_classifier_perf_test
```

Out[159…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.853623	0.761905	0.72	0.74036	0.91326

After running both tuning methods I got the same results. I will take the fisrt method to compile the resuts.

Tuned Gradient Boosting Classifier

In [160…

```
#GRID SEARCH METHOD FOR TUNING GC
gb_tuned = GradientBoostingClassifier(random_state = 1)

# Grid of parameters to choose from
gb_parameters = {"n_estimators": [100, 110, 120],

    "max_depth": [5, 7, None],

    "max_features": [0.8, 1]
             }

# Run the grid search with scoring f1 to balance
gb_grid_obj = GridSearchCV(gb_tuned, gb_parameters, scoring

gb_grid_obj = gb_grid_obj.fit(x_train, y_train)

# Set the gb_tuned_classifier to the best combination of par
gb_tuned_classifier = gb_grid_obj.best_estimator_

gb_tuned_classifier.fit(x_train, y_train)

# Model Performance on the test data
gb_tuned_classifier_perf_test = model_performance_classifica
gb_tuned_classifier_perf_test
```

Out[160…

	Accuracy	Precision	Recall	F1-score	ROC AUC
0	0.848551	0.741772	0.7325	0.737107	0.907847

The best model was since the beginnig, before manually tuning, Gradient Boosting, and after the tuning with gridsearch, between the Decision Tree and the Tuned Random Forest (RF2) and the Tuned RF with gridsearch, I can see that ***the Gradient Boosting got better performance metrics than DT and RF/RF2, much more balanced between precision and recall***, giving precision a little bit more importance, which means that if the business use this model, would prioritize in focusing in leads that are worthed, reducing the false positives and do not wasting time in bad leads.

Final Model Decision after comparing all the models

In [161…

```
# Compare the performance of the models in the test set
models_test_comp_df = pd.concat(
    [
        dt_classifier_perf_test.T,
        dt_tuned_classifier_perf_test.T,
        bagging_estimator_perf_test.T,
        rf_classifier_perf_test.T,
        rf_classifier_perf_test2.T,
        rf_tuned_classifier_perf_test.T,
        ada_classifier_perf_test.T,
        grad_classifier_perf_test.T,
        gb_tuned_classifier_perf_test.T,
        xgb_perf_test.T,
    ],
    axis = 1,
)

models_test_comp_df.columns = [
    "Decision Tree Classifier",
    "TUNED Decision Tree Classifier",
```

```
    "Bagging Classifier",
    "Random Forest Classifier",
    "TUNED Manually Random Forest Classifier 2",
    "TUNED Random Forest Classifier",
    "Ada Boost Classifier",
    "Gradient Boosting Classifier",
    "TUNED Gradient Boosting Classifier",
    "XG Boost Classifier"]

print("Test performance comparison the includes 4 tuned mode

models_test_comp_df.T
```

```
Test performance comparison the includes 4 tuned models:
```

Out[161…

	Accuracy	Precision	Recall	F1-score	ROC AUC
Decision Tree Classifier	0.797101	0.638889	0.6900	0.663462	0.770347
TUNED Decision Tree Classifier	0.837681	0.714634	0.7325	0.723457	0.887140
Bagging Classifier	0.839855	0.745205	0.6800	0.711111	0.881677
Random Forest Classifier	0.843478	0.747312	0.6950	0.720207	0.902856
TUNED Manually Random Forest Classifier 2	0.823913	0.654832	0.8300	0.732084	0.906370
TUNED Random Forest Classifier	0.853623	0.761905	0.7200	0.740360	0.913260
Ada Boost Classifier	0.807971	0.703927	0.5825	0.637483	0.869128
Gradient Boosting Classifier	0.844203	0.740260	0.7125	0.726115	0.908446
TUNED Gradient Boosting Classifier	0.848551	0.741772	0.7325	0.737107	0.907847
XG Boost Classifier	0.830435	0.715026	0.6900	0.702290	0.898661

Observations:

- **Accuracy**: For this case is not that relevant, so even when my rates are high, on or above 0.80 for all the models tested, I need to evaluate precision and recall as I have mentioned before to avoid loosing portencial customer or suggest the company to loose resources in leads that are not going to convert.

- **Precision and Recall** during previous steps I realized and commented that the performance metrics tell me that my model was not predicting well after the comparison between train and test metrics per each model applied. In the first run I would say it was totally overffited.

- For the **Decision Tree I improved the recall from 0,69 to 0.73 a decent amount after the tuning**

- For **Randon Forest** during the first run the recall was 1 for train and 0.69 for test, overfitted as well. After tunning manually and with gridsearch, the model improved the metrics improving recall and decreasing precision in the RF2, and improving both of the metrics after using gridsearch with score f1. **Between the 3 Random Forest lines we can see the trade-off between precision and recall and at the end a TUNED more balanced model**.

- **GRadient Boosting is my best balance model after tuning**. 0.74 preciosion. 0.73 recall and 0.73 f1. This model was not overfitted since the first run as I explained below the train metrics previously.

- **F1 score**: The best F1 metrics are in the Tuned Random Forest 0.740 and Tuned Gradient Boosting models 0.737. By

definition,this metric is the harmonic mean between precision and recall and could be useful if we just want to see the model performance under one single metric, in this case it helps to **balance the false negatives**, when my best model predict that a lead will not converted, but in real the lead get converted, and balance as well the **false positives** when the model predict that a lead will get converted, but in the end, the lead does not convert. Thinking that the data set is imbalance for the first run, this metric is useful for the customer prediction analysis and it is my preference variable.

- **ROC AUC** (Receiver Operating Characteristic - Area Under the Curve) is 0.91 which is near 1, theoretically my model is good enough to distinguish between the leads that will convert vs. the leads that will not convert to paid customers. BUT, this metric is more applied for general comparison and does not tell us how the model performs at the threshold we will actually use. The model could have high AUC, as in all of these cases, but still have poor recall or precision, and that is not what we need for this specific case when we try to achieve conversion of the leads. So this metric will not be considered as a metric for the final comparison of the best models.

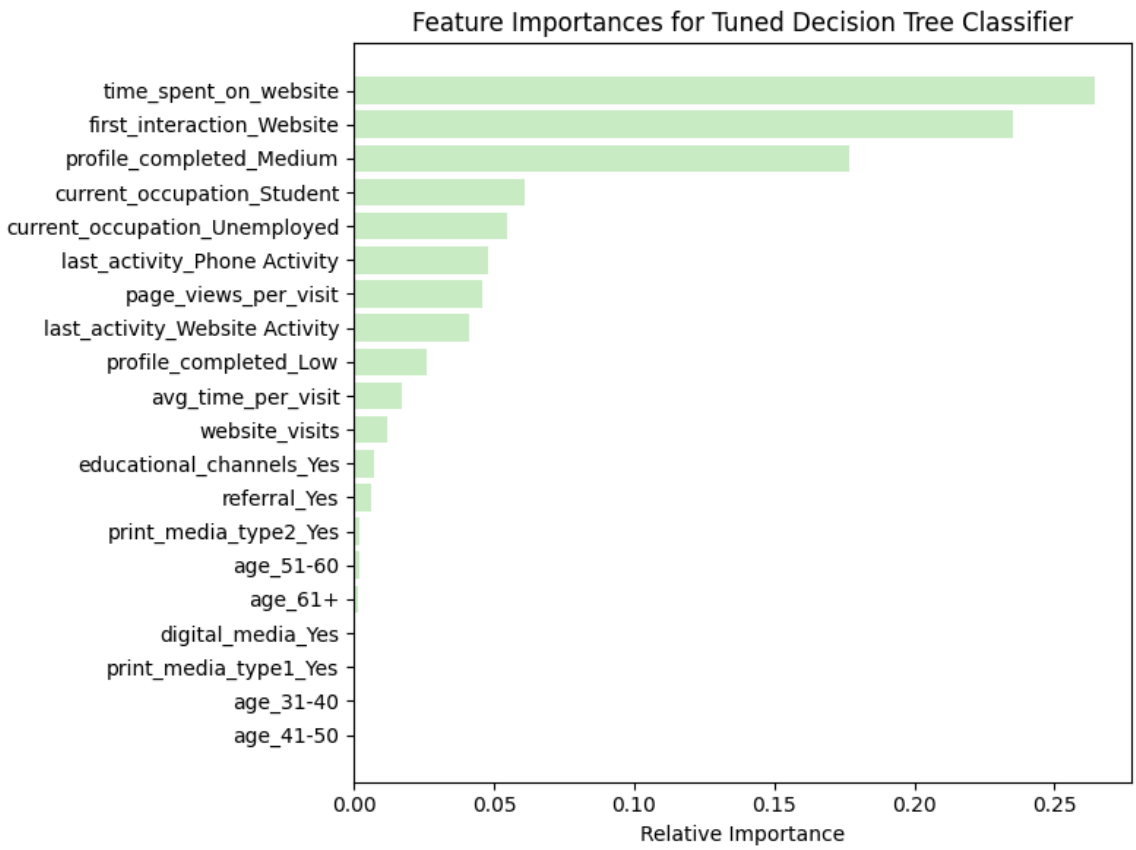
Feature Importance for the Tuned Models DT, RF and GB

These are going to be the features that contribute the most to the model's predictions. We will see which are the features that the 3 models have in common and get the conclusions for the profile of leads. I will comment below.

In [162…

```
# Plotting the feature importance for the best model (dt_tur
features = list(x.columns)
```

```
importances = dt_tuned_classifier.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize = (8, 6))
plt.title('Feature Importances for Tuned Decision Tree Class
plt.barh(range(len(indices)), importances[indices], color =
plt.yticks(range(len(indices)), [features[i] for i in indice
plt.xlabel('Relative Importance')
plt.tight_layout()
plt.show()
```

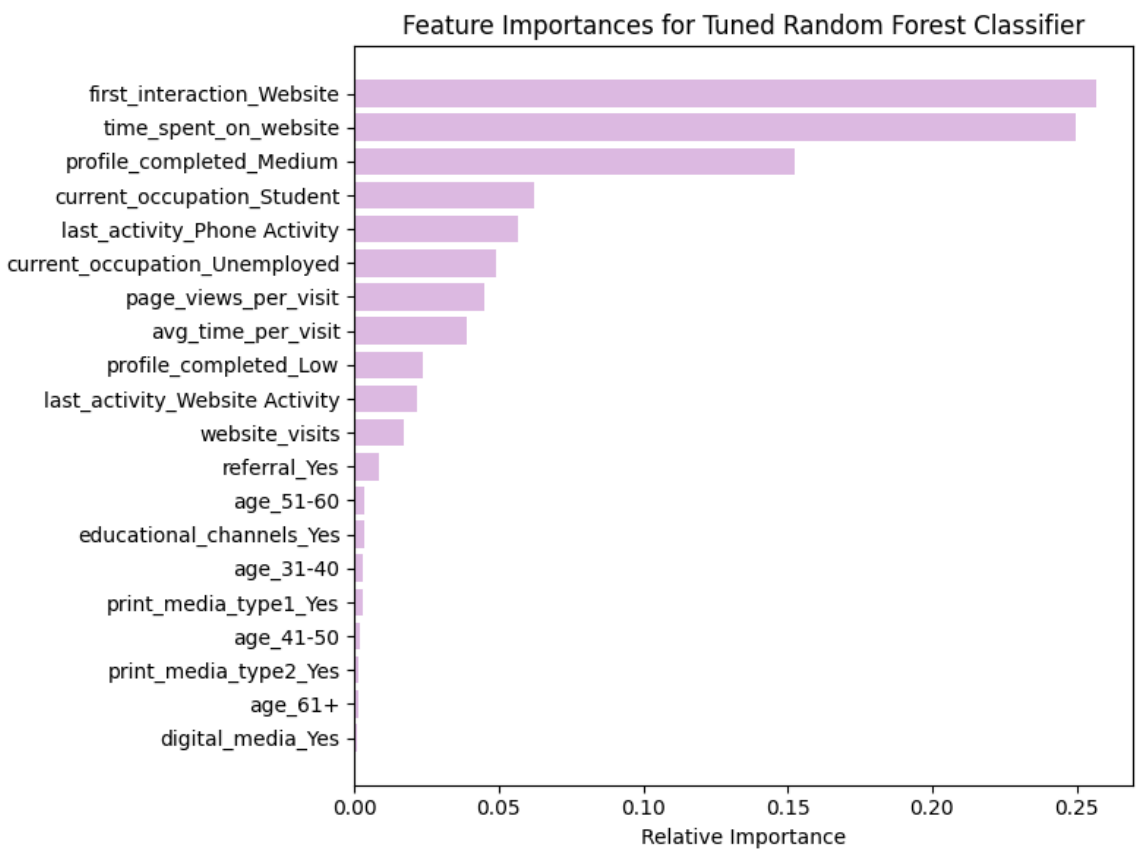


For the DT tuned classifier the most important variables for the prediction are:

- Time spent on webiste
- First interaction / Website
- Profile completed / Medium
- Current Occupation / Student The fith variable is Current Occupation/Unemployed

In [163…

```
# Plotting the feature importance for the model (rf_tuned_cl
features = list(x.columns)
importances = rf_tuned_classifier.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize = (8, 6))
plt.title('Feature Importances for Tuned Random Forest Class
plt.barh(range(len(indices)), importances[indices], color =
plt.yticks(range(len(indices)), [features[i] for i in indice
plt.xlabel('Relative Importance')
plt.tight_layout()
plt.show()
```



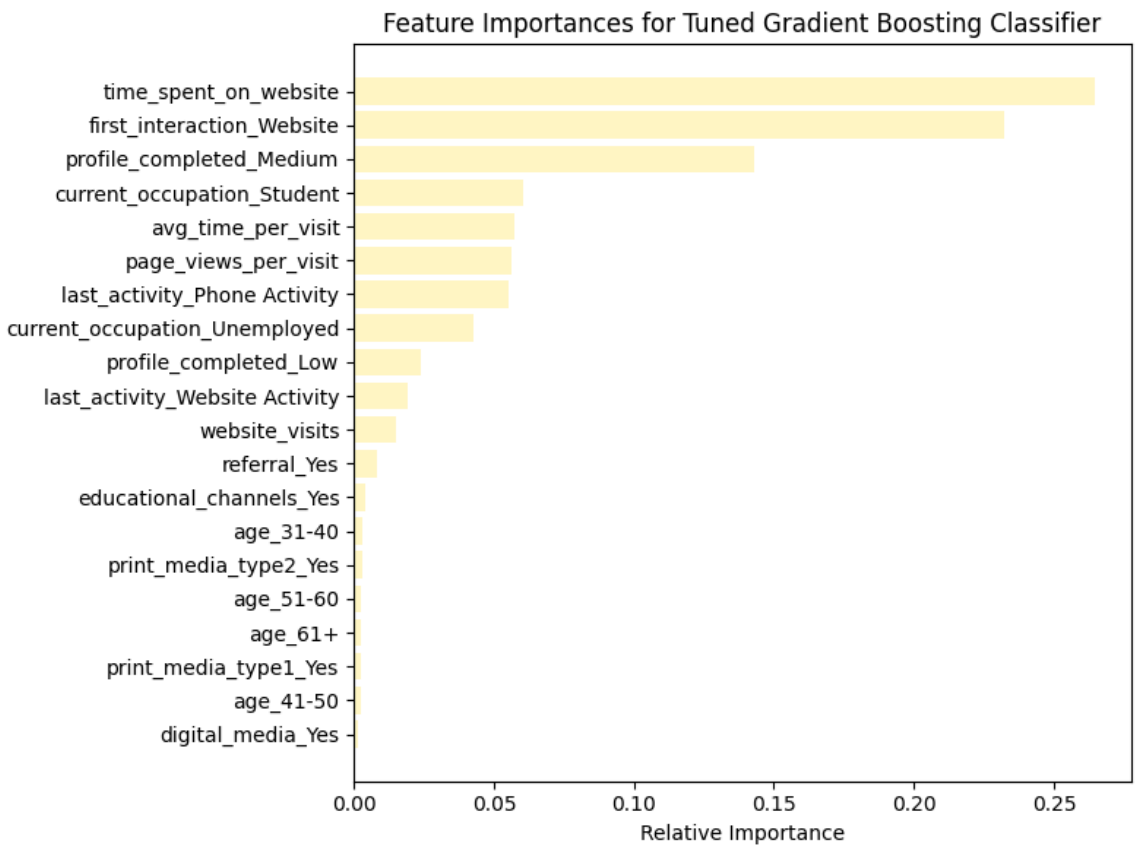
For the RF tuned classifier the most important variables for the prediction are the same that in the DT tuned, but the first 2 variables inverted in order:

- First interaction / Website
- Time spent on webiste
- Profile completed / Medium

- Current Occupation / Student The fith variable is Last Activity/Phone

In [164…

```
# Plotting the feature importance for the model (gb_tuned_cl
features = list(x.columns)
importances = gb_tuned_classifier.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize = (8, 6))
plt.title('Feature Importances for Tuned Gradient Boosting C
plt.barh(range(len(indices)), importances[indices], color =
plt.yticks(range(len(indices)), [features[i] for i in indice
plt.xlabel('Relative Importance')
plt.tight_layout()
plt.show()
```



For the GB tuned classifier the most important variables for the prediction are the same to the Decicion Tree tuned model, and in the same order:

- First interaction / Website

- Time spent on webiste
- Profile completed / Medium
- Current Occupation / Student

Curiously in this model the 5th variable is the created variable, average time per visit.

Profile of leads

A Profile of leads which are likely to convert contains the variables with the strongest predictive power, I would focus on the first 3 in the graphs above, taking in consideration that the size of the bars gets reduce from the fourth to the end:

- ***The time_spent_on_website and first_interaction through the website are the strongest variables of the model to predict the convertion***. We can tell that the user experience on the website can be more informative, engaging, or user-friendly for leads than the mobile app, leading to a higher likelihood of conversion. The website might offer a more comprehensive overview of programs, testimonials, pricing, chat with me option, call me button that works well and facilitates fast phone contact, download the programs once the lead register some personal information, and features that are working well in helping leads make a more informed decision, accessible contact, and finally encouraging the further advance towards the enrollment process. The ***time that a lead spent in the website** looking around, consuming the information and using the features reflect the level of interest in joining a program. All of the features I mentioned are assumptions, if those features are not present yet, these could be a point of improvement for the website, but given the fact that the website interaction works well, I am

assuming the website is strong in presence, content and features.

- ***The profile_completed_Medium tells about a considerable level of interest and level of engagement from the lead, reflected in the information they share in their profile, and it is a strong indicator of their conversion potential***. Nevertheless the decision tree had this variable minus 0.5, which means that when the profile_medium is 0 is going to convert, so it will match with high which had better convertion rate, or even low profile completion, this last one was not good to convert. In general the level of completion would show that the lead is willing to invest time and share its personal information to be contacted and get more information, revealing strong interest which is a crucial step towards a enrollment decision.

There are other variables that appear in the models I would like to comment about:

- Surpringsingly, **the current_ocupation variable appear as the 4th important feature for the profile. **It happens the same situation with minus 0.5 it means that when the ocupation is not student it will convert more. Student ocupation variable is present in the 3 models and Unemployed ocupation is in 2 of the models. I would think that in general the ocupation has a moderate impact on the likelihood of lead becoming a paying customer. It could be that leads unemployed try to dedicate their free time in upskilling or reskilling, and at the same time pave the way to land a new job position. About the students category, as they have very low level of convertion in the dataset, we would need to recommend to focus specific or limited resources in capturing students to follow their next academic path in

Extralearn and verify in practice if the younger profiles convert, but adjusting some business rules to accompany this new strategy as a pilot project, since with the data given, stuadents do not convert to paid customers.

- Regarding the **last_interaction by phone call, it is moderately relevant since appears in 2 models as part of the best 4 or 5 features**. As we see in the bivariate analysis, leads that talk by phone are very little in comparison with the email interaction for example. I would assume that leads that are interested, look for help call the support center service. The call center have to take advantage of this interested leads and follow the steps to capture the lead and enroll it. I would revise this option since maybe the call center has not being doing a good job because the interaction by phone is reduced, and the email interaction is a stronger indicator to convert.

- Contrary to what I thought at the beginning about the referral variable, here we see that in real, after the whole analysis is performed, ***the referral variable has very little influence on the conversion in this model*** at the present. I attribute this lack of power to the small quantity of leads that come from referrals. Maybe increasing the leads that come from this category, Extralearn can increase the convertion based on this variable and become in the future one of the profile variables.

- ***The variables related to the sourcing/origin of the lead through printed media like print_media_type1, print_media_type2, educational_channel and digital_media have no real influence on the conversion in the model***. This suggests that, once a lead arrives on the platform, the channel through which they initially arrived has

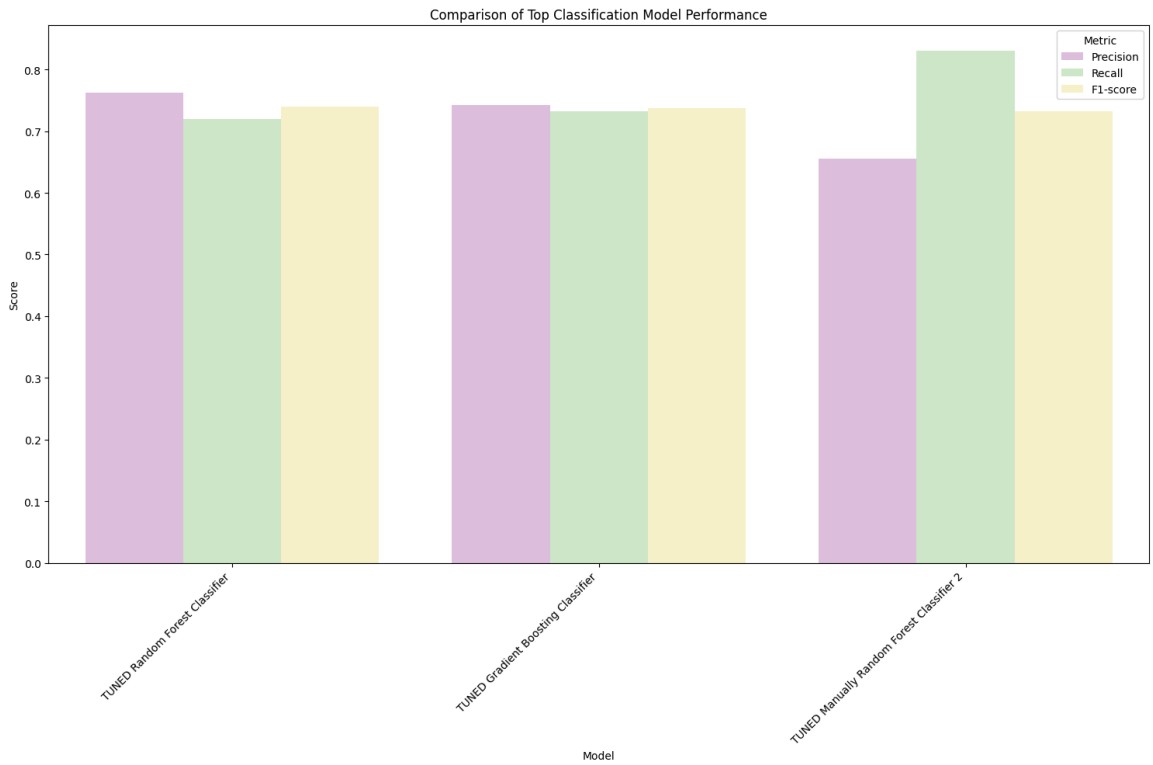
less weight in their final conversion decision than their behavior, interaction and engagement that the leads can get from the features website provides.

Top Model Performance Comparison

We are going to visualize the key performance metrics of our best models to get a clearer comparison.

In [165…

```
# Select top models for comparison. I am going to plot just
top_models = models_test_comp_df.T.sort_values(by='F1-score'

# Metrics to plot just 3 more important for the case
metrics_to_plot = ['Precision', 'Recall', 'F1-score']

plt.figure(figsize=(15, 10))
sns.barplot(data=top_models[metrics_to_plot].reset_index().m
            x='index', y='value', hue='variable', palette=['
plt.title('Comparison of Top Classification Model Performanc
plt.ylabel('Score')
plt.xlabel('Model')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Metric')
plt.tight_layout()
plt.show()
```



Observations:

- Based on the results obtained after comparing all of the tuned models, **Gradient Booting Clasifier is the best-performing model, and as we can see in the graph above is the more balanced model between precision and recall, which is good if we do not know the real business costs and priorities/strategy for the period, or like in this case of study, where the data is very limited in terms of enrollment fees or marketing cost per lead per channel**.

- ***If I would have to choose between the Decision Tree and the Random Forest models, the best model is the Tuned Random Forest**. At the beginning, before tuning, no model was enoughly good, all of them were overfitted as I already explained in the previous steps.

- **Tuning the hyperparameters** of the best models with gridsearch was a crucial/important step that helped **to optimize the model's performance** by finding the best set

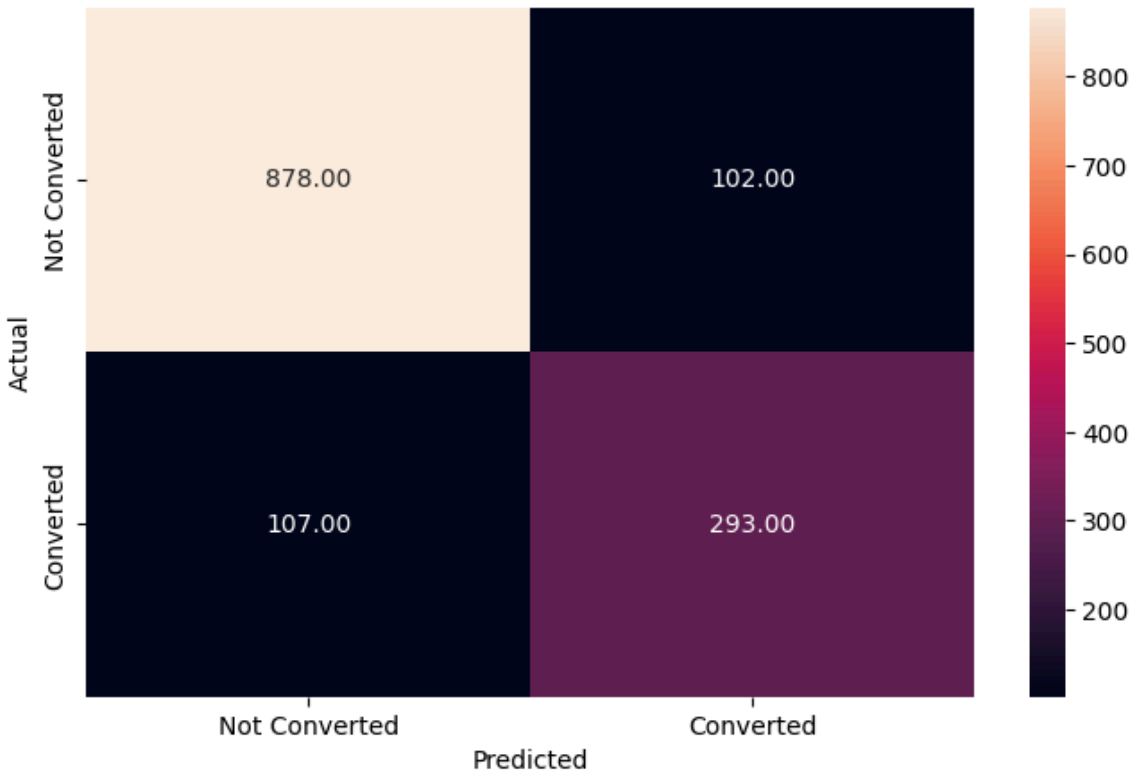
of hyperparameters that work well for the dataset. The manual tuning that I did for the Randon Forest 2, incrased a lot the recall which could be good as well if the business needs were more focus on only capturing the TP leads, but paying the price of decreasing the precision, not really achieving the balance.

- **Depending of the business needs, any of these 3 models and features is good enough to improve the marketing operations, lead capture and convertion.**

In [166…

```
# Function to print the classification report

def metrics_score(actual, predicted):
    print(classification_report(actual, predicted))
    con_matrix = confusion_matrix(actual, predicted)
    plt.figure(figsize = (8, 5))
    sns.heatmap(con_matrix, annot = True,  fmt = '.2f', xtic
                yticklabels = ['Not Converted', 'Converted']
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()

print('Classification Report and Confusion Matrix for Tuned
metrics_score(y_test, gb_tuned_classifier.predict(x_test))
```

```
Classification Report and Confusion Matrix for Tuned Gradient
Boosting Classifier:
              precision    recall  f1-score   support

           0       0.89      0.90      0.89       980
           1       0.74      0.73      0.74       400

    accuracy                           0.85      1380
   macro avg       0.82      0.81      0.82      1380
weighted avg       0.85      0.85      0.85      1380
```



***Observations for the Confusion Matrix** The plot shows the actual versus predicted counts:

- True Negatives (Top-Left): Leads correctly predicted as 'Not Converted'. ***Precision for class 0 (Not Converted): count 878 out of 980, meaning 89% of leads predicted as 'Not Converted' were actually 'Not Converted'**.

- True Positives (Bottom-Right): Leads correctly predicted as 'Converted'. **Recall for class 1 (Converted): count 293 TP out of all the actual positives (400 = 293+107), meaning 73% of actual 'Converted' leads were correctly identified, almost 3 out 4 positive predictions are right, my model is detecting the positives now. This is way better than the 46% that I have gotten initially in my first overfitted model**

- False Negatives (Bottom-Left): Leads incorrectly predicted as 'Not Converted' (actual was 'Converted'). **Precision for class**

1 (Converted):107, 293 TP out of all the predicted positives (395 = 293+102), meaning 74% of predicted as 'Not Converted' were actually 'Converted'

- False Positives (Top-Right): Leads incorrectly predicted as 'Converted' (actual was 'Not Converted'). **Recall for class 0 (Not Converted): 102, leads predicted as Converted that actual 'Not Converted'.**

- F1-score for class 1 (Converted): 0.74, which is a balanced measure of precision and recall.

- Accuracy: The overall accuracy of the model is 0.85 (85%).

The model shows good overall performance, with a reasonable balance between precision and recall for the 'Converted' class.

Insigths

- The Preliminary Data Analysis gave us from the beginning certain clues about the variables that play an important role in the prediction. Even when we apply simple statistical analysis, we discovered the behavior of the leads related to each variable like interaction by website and time_per_visit that later were reassured by the most important features useful for the prediction.

- The first interaction by website is a good predictor, and it is more related to the nature of the business and the way that people interact at the present to find online content on the web than in any other place. The website usually has a strong presence and is the first face of the company for the leads. Supported by the numbers, the leads that converted to paid customers are 42,66% (1,375/4,598). Out of the 1,375, a

quantity of 1,157 (84%) a high percentage had their first interaction by the website.

- Curiously, the number of website_visits, since the beginning was not a clear factor of the prediction, telling us that the even when a lead visit the website just 2 or 3 times, the most important fact is not the quantity of visits, since it is in average 3 for positive/negative outcomes, but it is the time that the lead invest in its research about the offer/programs.

- The time_spent_on_website variable was on several ocassions during the analysis targeted as an important predictor. The interest is shown clearly by the time the people invest in reading and researching. This can be clearly seen in the distribution plot wrt target and at the end in the decision tree and random forest model results.

- The last interaction of the customers, last_activity variable, for me was not a clear predictor since given the dataset that we have, we can not really see in what stage is the lead. The leads are in constant evolution, and in educational field, sometimes the people take long time or at least several months to decide what to do, if enroll or not. At the preliminary analysis having a last_activity variable without any other variable that talks about the stage in the pipeline that the lead is, was pretty difficult that this variable is used as predictor, BUT after running the decision tree and random forest model my preliminary conclusion was not proven. The phone activity might serve as a variable to confirm the paid customer status.

- ***For customer-prediction models, not a single metric is "the most important" in all cases, it is going to depend on what business consequences the companies care more about. The cost of the model mistakes need to be***

identified to take a final decision on what metric we are going to focus on, BUT I would recommend to consider ***F1 is a good metric to simplify the problem** when getting a real cost per customer lost, or the real cost of an specific campaign per lead was too difficult or time consuming to calculate. Usually the small companies do not have robust cost control systems in place to define the exact cost per business object.

Actionable Business Recommendations

1. Analyse the income in terms of the profile of leads that more convert. Having already identified the main 3 features of the profile, Extralearn can filter the enrolled leads that match the profile out of the whole population of customers to determine if in fact, this group of customers represent at least more than the 50% of the income. This would be a test for the profile determined previously. If the profile support the income in good proportion it means Extralearn can focus on assuring that those type of leads are well taking care during the lead lifetime, by implementing aditional steps or performance indicators per stage of the funnel. If the current income is not really supported by the profile, which can be another situation with much more reasons involved given the time variations, changes in market or changes in offer, it would be good to determine the best combination of features taking the income variable as part of the dataset, the features of clients that in fact make the business profitable.

2. In general, for this case, all of the sources of leads, represent by itself a very few portion of the total of the leads that became paid customers within each category. In order to convert more leads to paid customers, first we need to

increase the quantity and the quality of leads. I would recommend to the marketing department to diversify even more the sources of leads by:

- Defining or redefining the strategy in terms of the offer, prices and align the marketing objetives to achieve the strategy taking as well in consideration the income analysis from point 1. The programs available in their offer vs. the type of leads that they are going to focus on to maximize their income, for example with current occupation students. In the data we saw the paid customers were mainly people older than 40 years old and around 55 years old, mostly professionals or unemployed. ExtraLearn could start a pilot project on getting younger leads/students since the problem narrative include upskill, not just reskill.

- Promoting their programs in educational fairs organized by local governments where the people go specifically looking for programs online.

- Analyze and implement a good price strategy that allows young or unemployed people to join their programs.

- Getting databases from specific high schools that are aligned to their new iniciative of getting younger leads/ client target to send focus campaigns for those kids that look for online education in the areas where ExtraLearn has an academic offer.

- Partnering with recognized education counselors or educational consulting companies to offer an incentive program that articulates academic guidance and support to clients/professionals and unempoyed leads more than 40 yo

mainly, or students of all ages pointing out the ExtraLearn programs available online.

- Sending targeted messages to users, professionals, in Linkedin filtered by specific job titles that match with the ExtraLearn academic offer.

3. Once the leads are in the pipeline, ExtraLearn needs to **improve the capacity to convert, by creating new ways to keep the lead hot**, assuring easy access to information, resolving doubts about the programs offered, providing a fast enrollment process and payment flexibility. All the stages are important to keep the lead hooked until it takes the final decision of enrollment, for example:

- Specific follow up by email and phone, double the efforts, for the leads that spend more than 5 minutes on the website per visit and visit the website more than 3 times which is the average. This is a recommendation made from the combination of some of the previous analysis.

- Tuition calculators that show real costs after financial aid, simulators can be added to the website to give a fast idea of the investment, useful for the people that do not want to talk until they know how much it will cost.

- Offering flexibility and different ways to pay depending on the price of the program.

- Offering free course modules to showcase the value of their programs can be a good option for the difficult leads, the ones that are interested but need a push to take the decision.